# Organización del Procesador

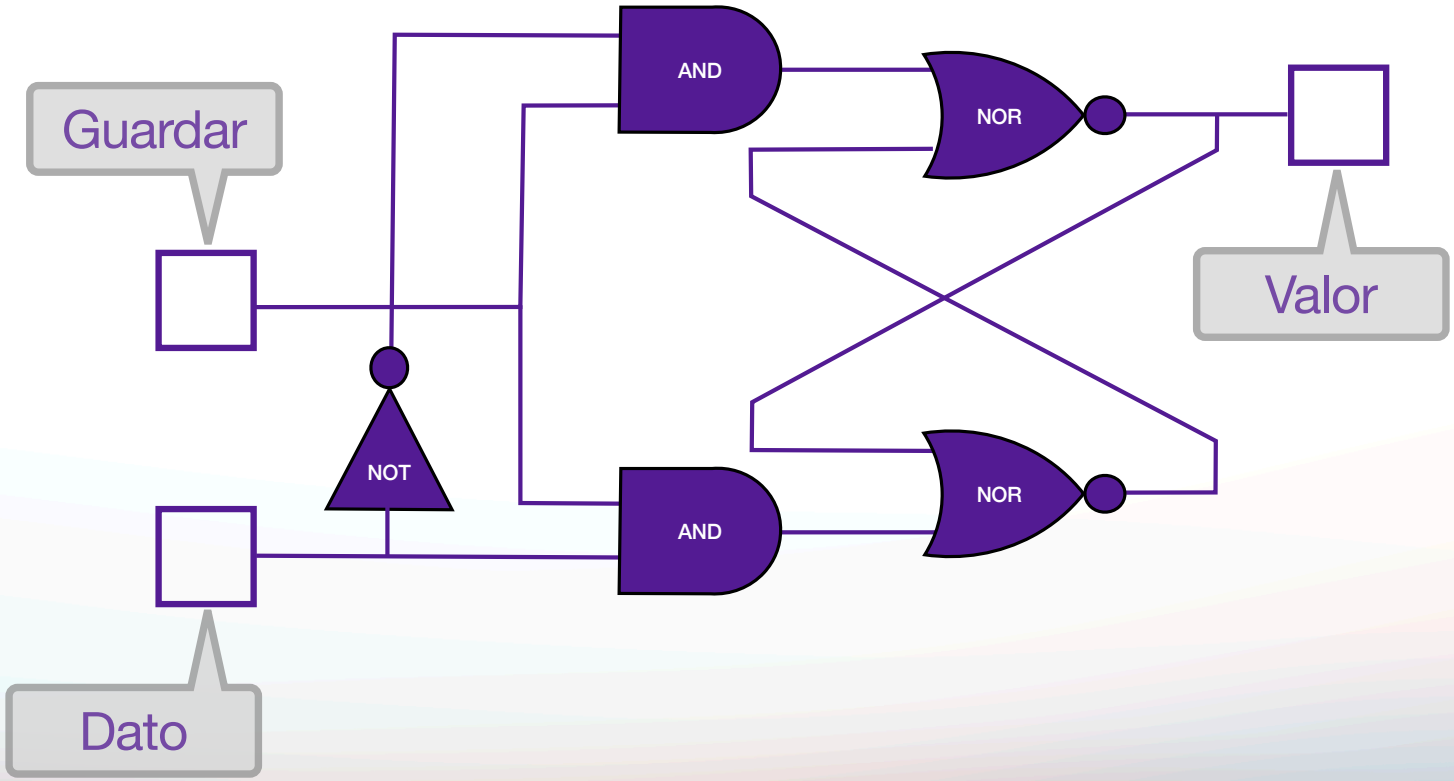
## Cómo computar utilizando la electricidad
## Operaciones Aritmético-Lógicas

Departamento de Computación - UNRC

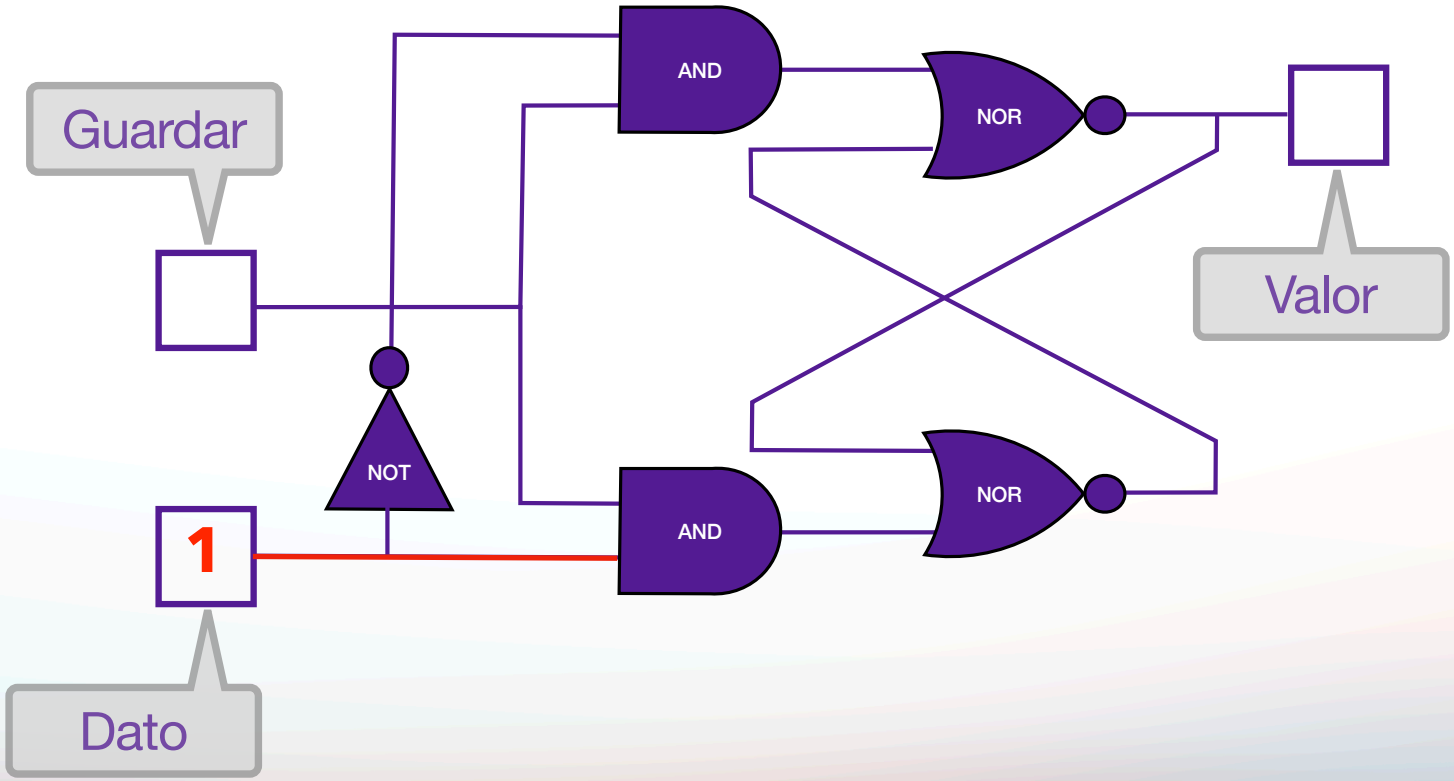
## El camino a recorrer

- Un poco de Historia y Sistemas Numéricos
- Introducción a la Electrónica
- Representación de Información
- **Cómo computar utilizando la electricidad**
- Evolución y Funcionamiento abstracto de una computadora
- Assembly X86
- Micro-programación (cómo fabricar un procesador)
- Eficiencia
  - Pipelines
  - Memoria Caché
  - Memoria Virtual

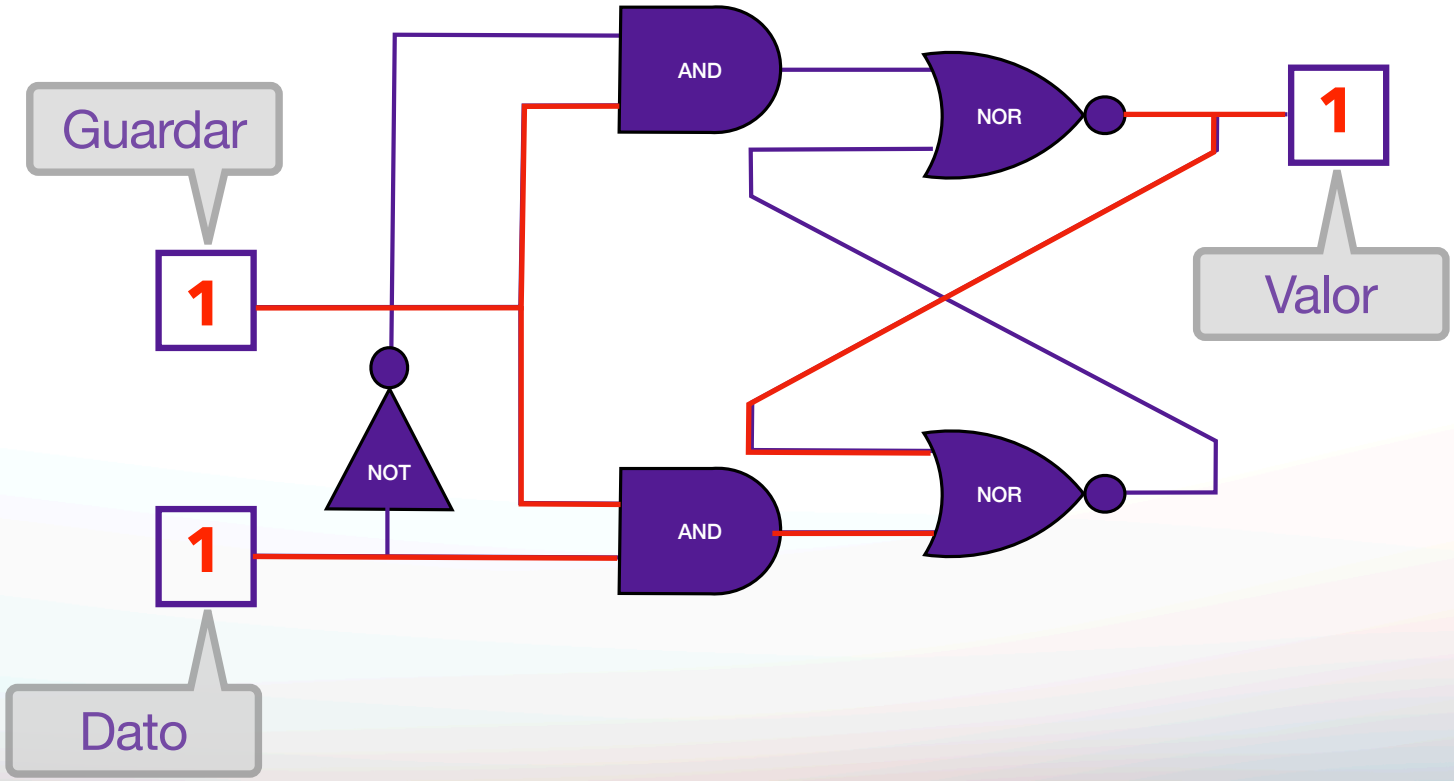
# Cómo podemos controlar la corriente eléctrica - Un bit de memoria



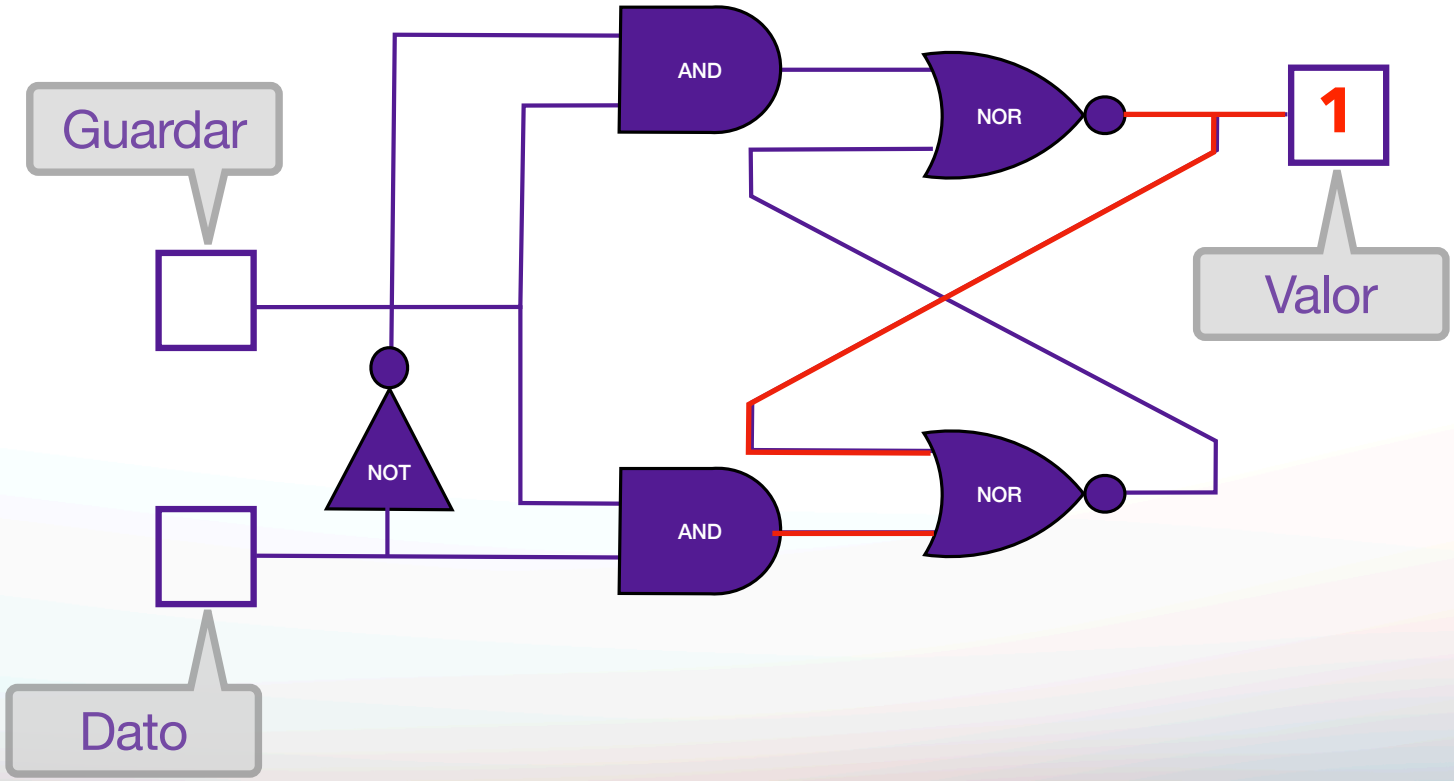
# Cómo podemos controlar la corriente eléctrica - Un bit de memoria



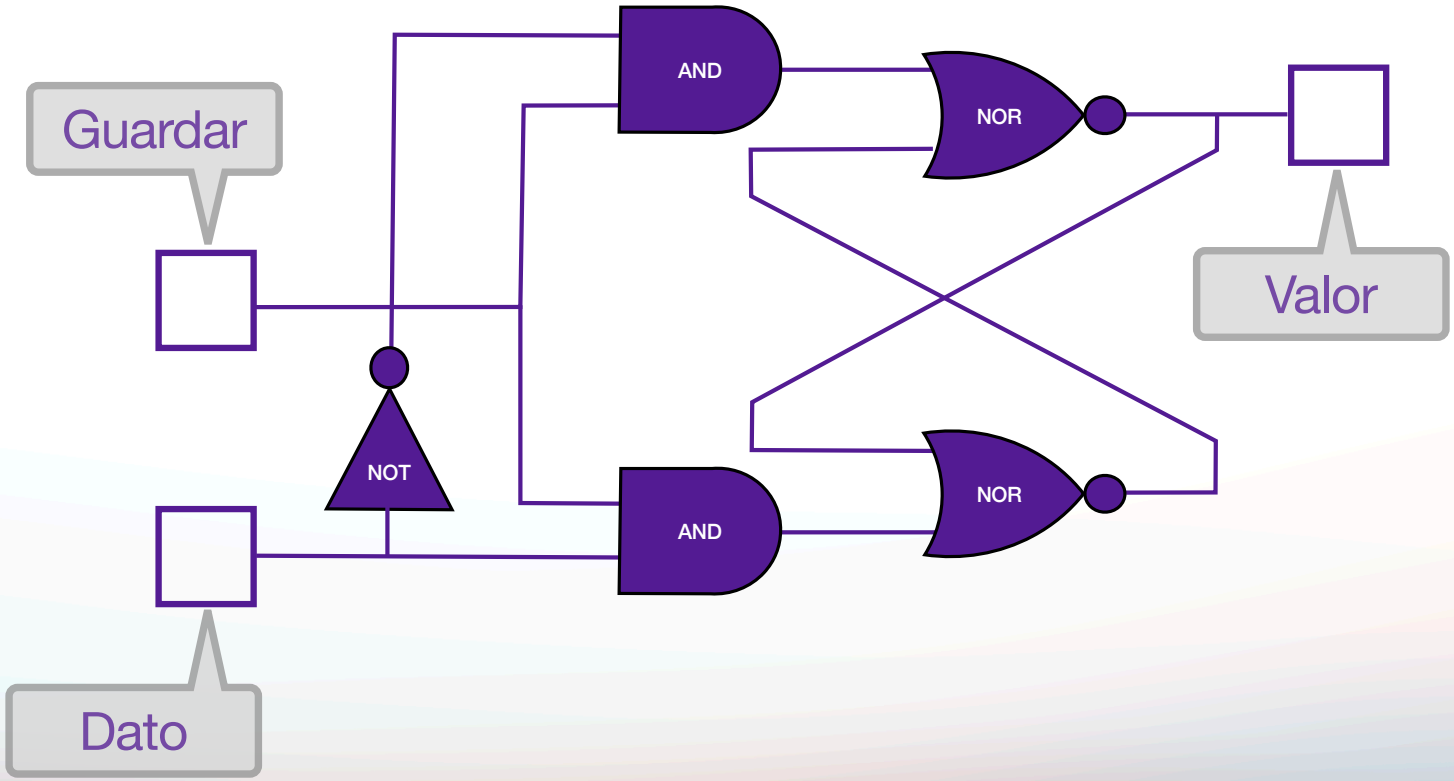
# Cómo podemos controlar la corriente eléctrica - Un bit de memoria



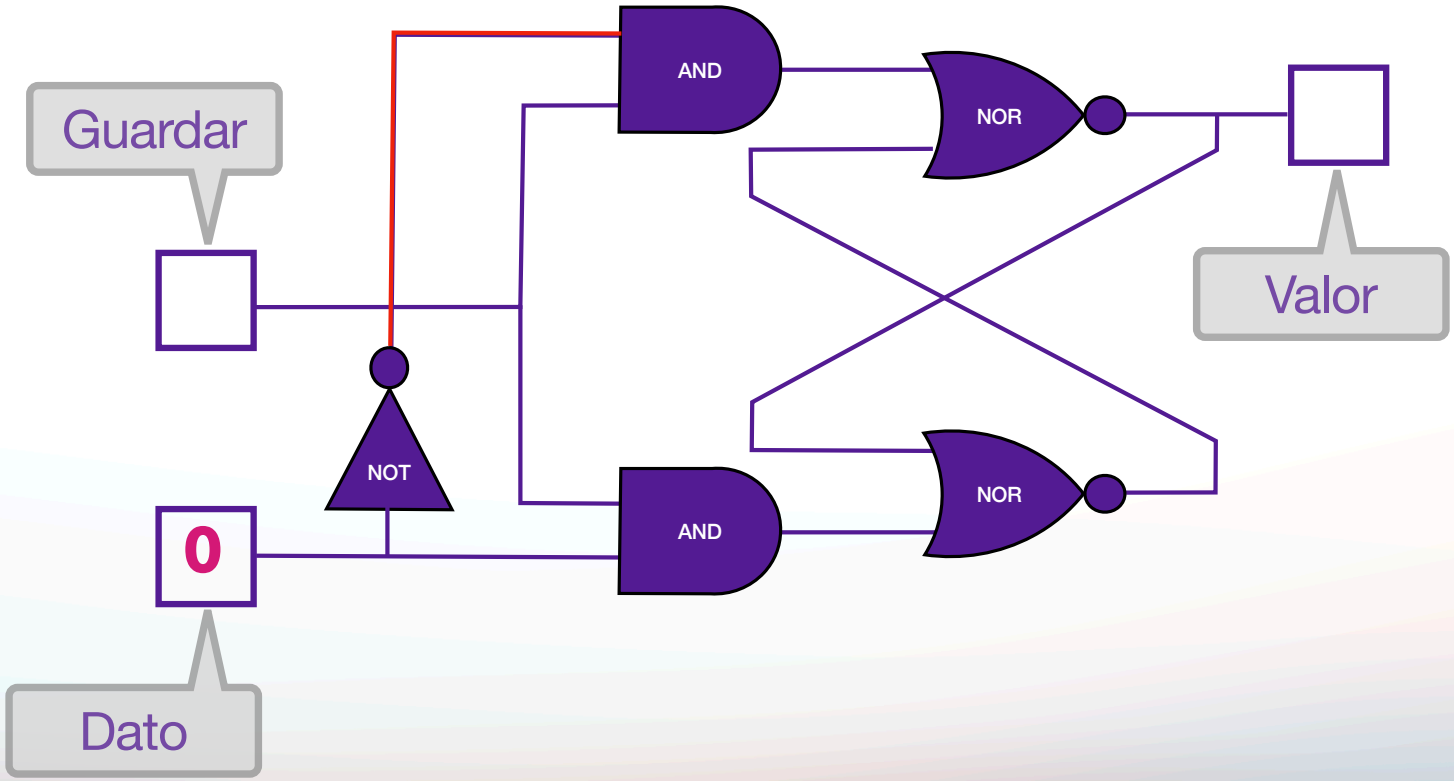
# Cómo podemos controlar la corriente eléctrica - Un bit de memoria



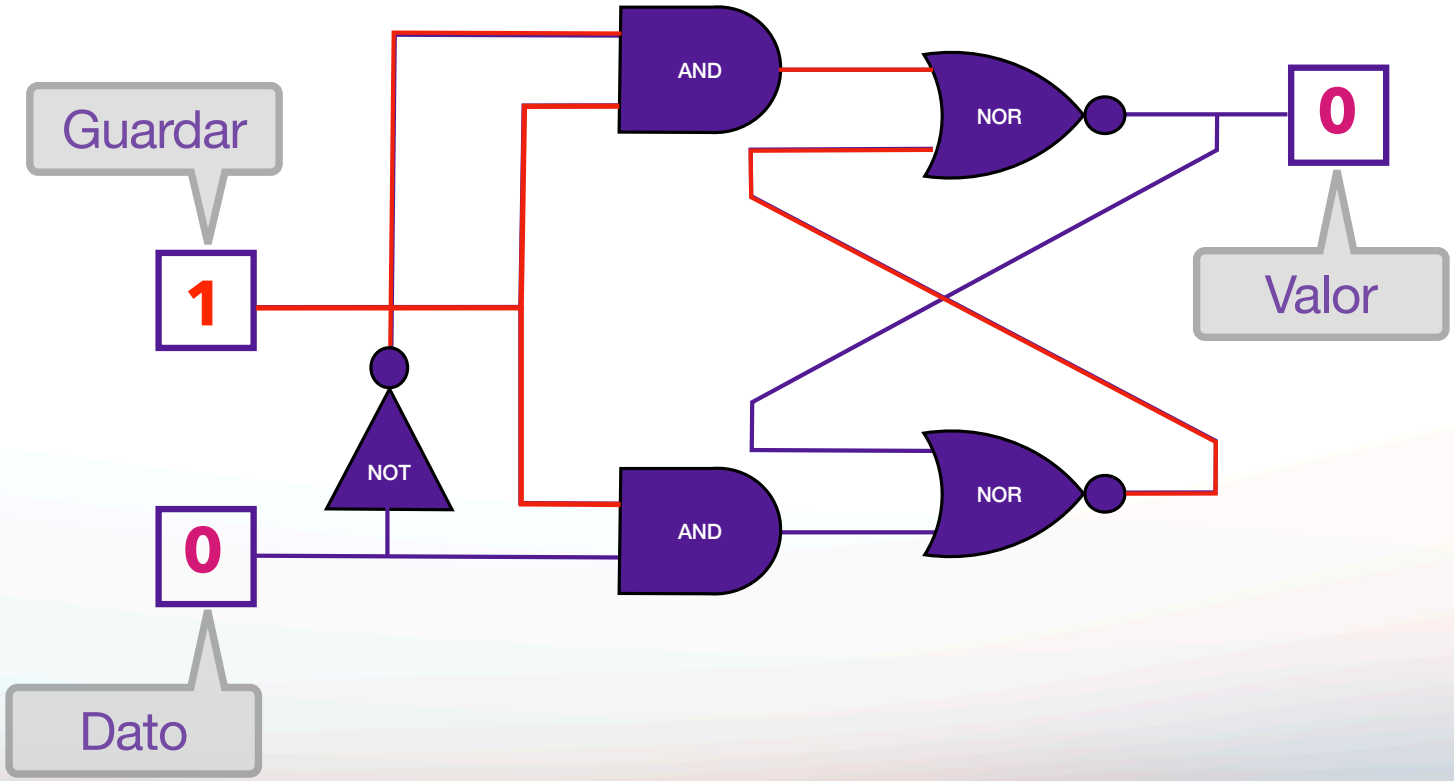
# Cómo podemos controlar la corriente eléctrica - Un bit de memoria



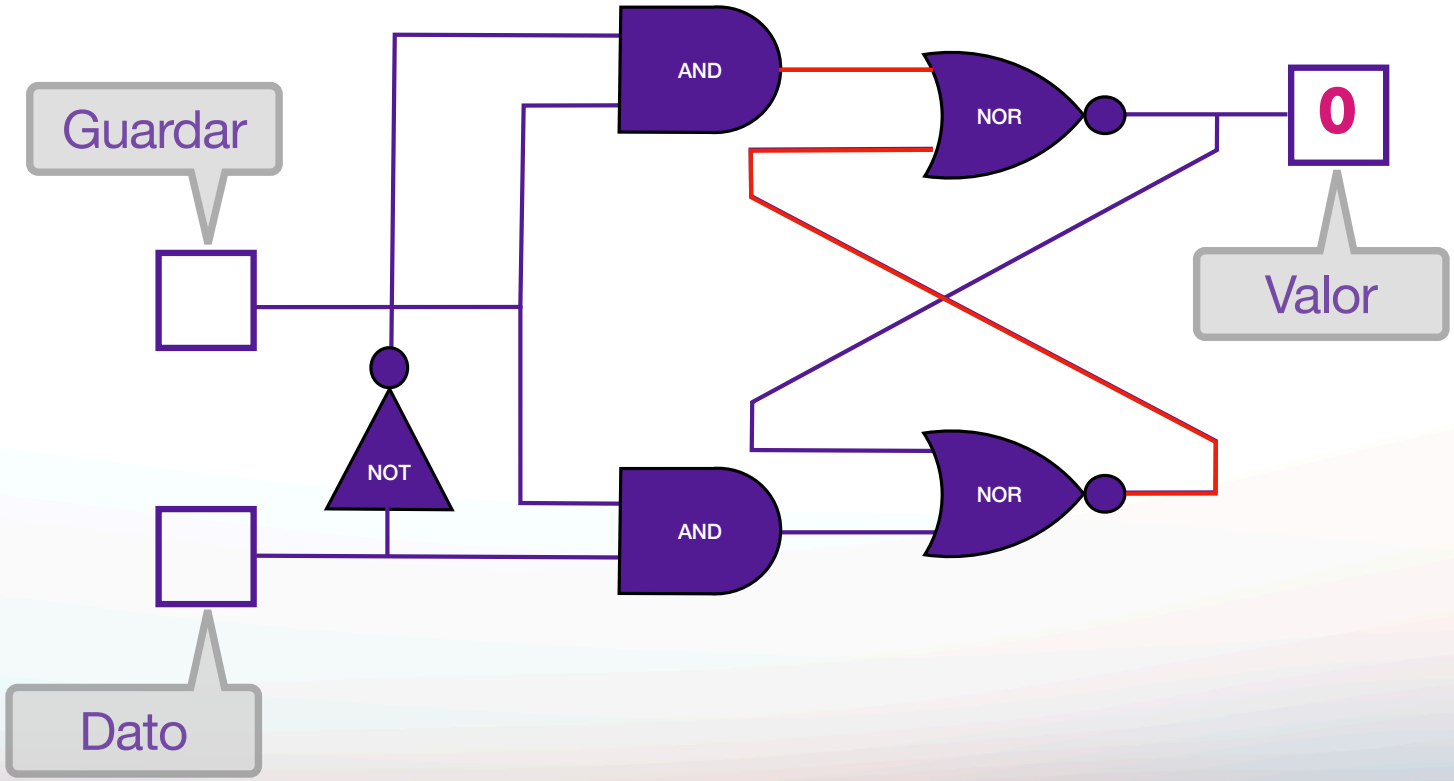
# Cómo podemos controlar la corriente eléctrica - Un bit de memoria



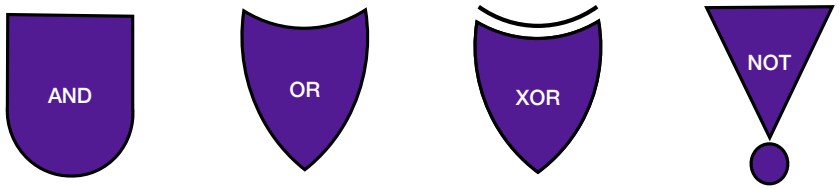
# Cómo podemos controlar la corriente eléctrica - Un bit de memoria



# Cómo podemos controlar la corriente eléctrica - Un bit de memoria

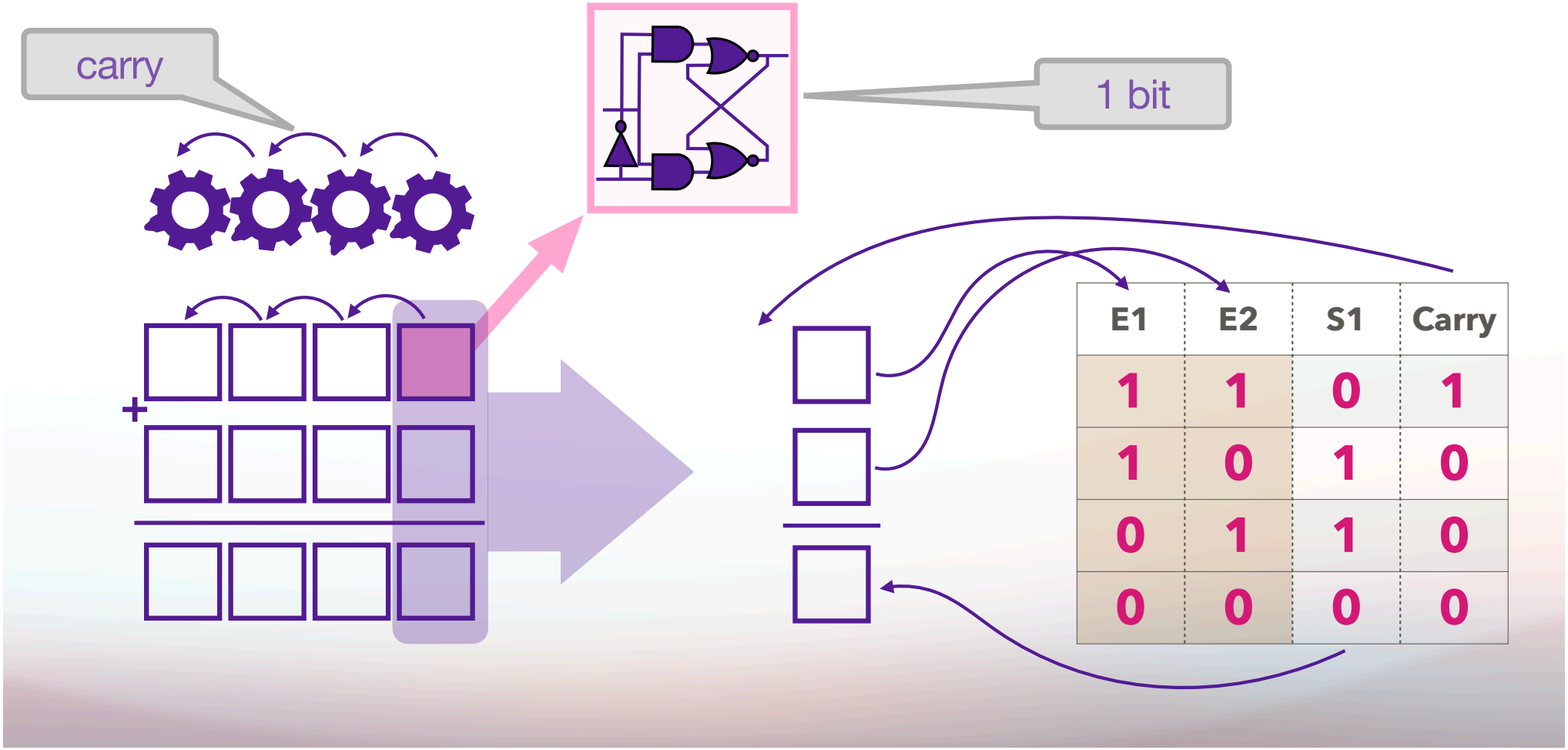


# Cómo podemos controlar la corriente eléctrica - Compuertas Lógicas



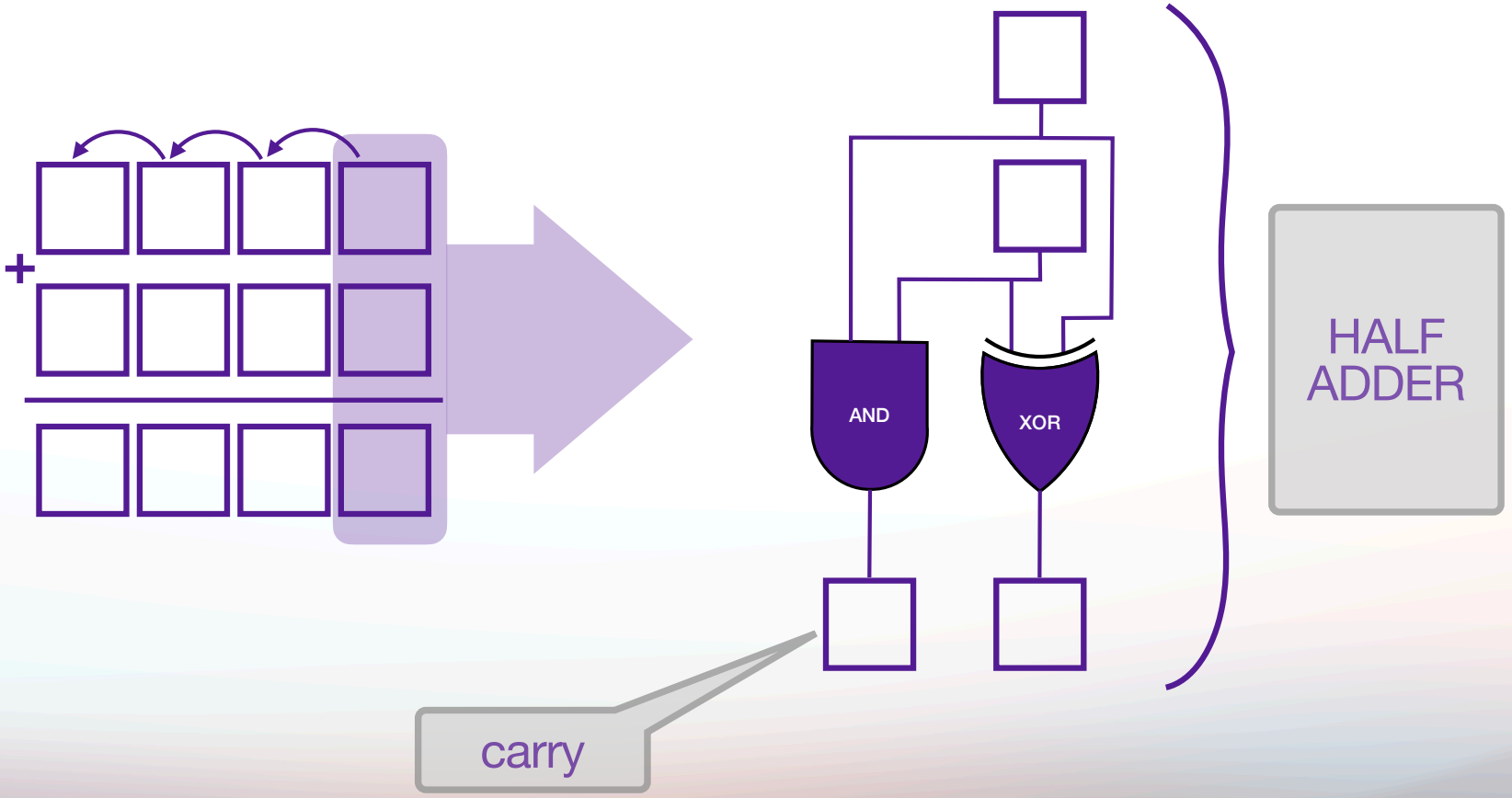

| E1 | E2 | E1 $\wedge$ E2 | E1 $\vee$ E2 | E1 $\veebar$ E2 | $\neg$E1 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 | |
| 0 | 1 | 0 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 0 | |

# Comencemos - Rehaciendo la PASCALINA con electricidad

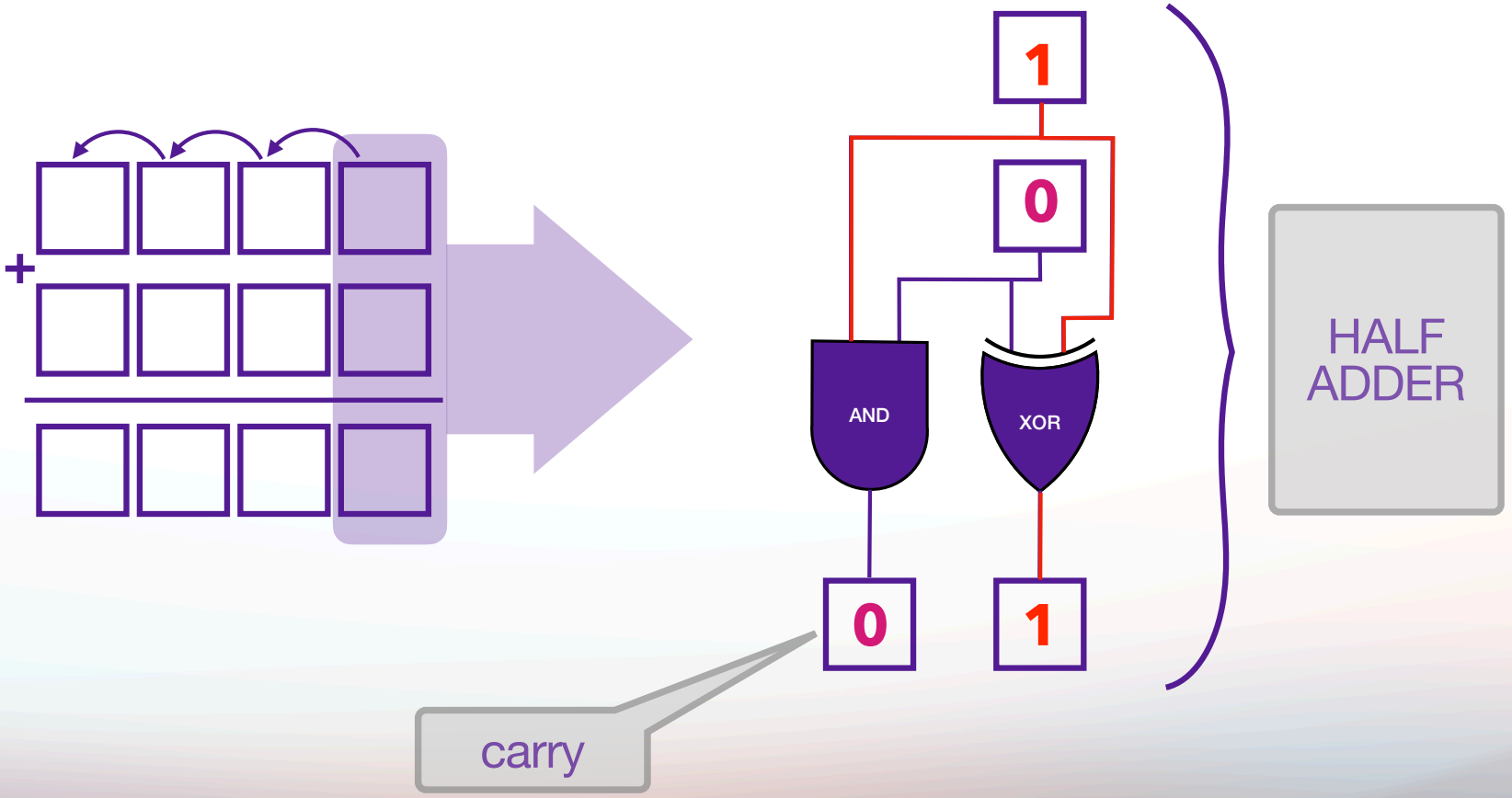



| E1 | E2 | S1 | Carry |
|---|---|---|---|
| 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 |

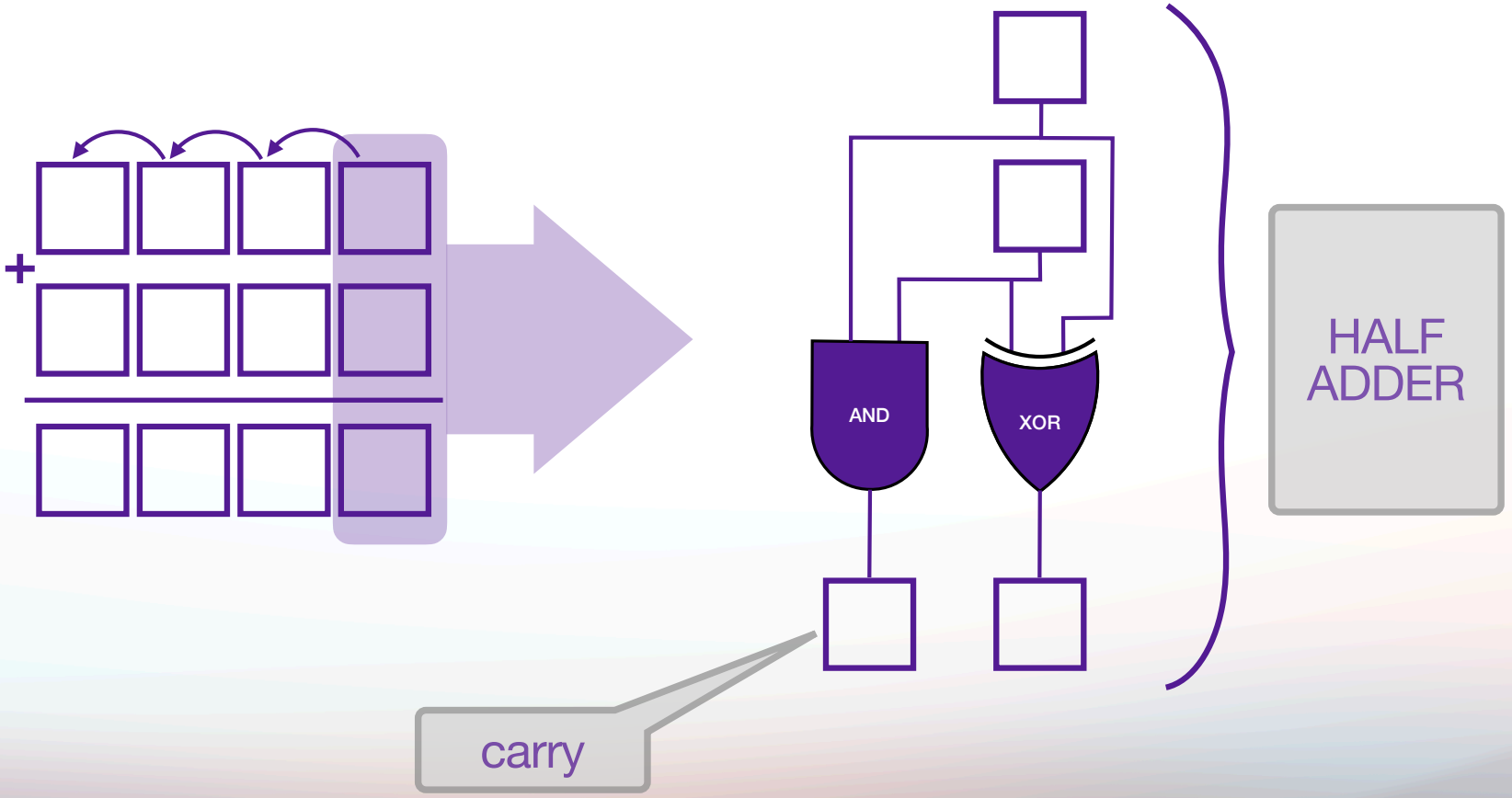
# Comencemos - Rehaciendo la PASCALINA con electricidad



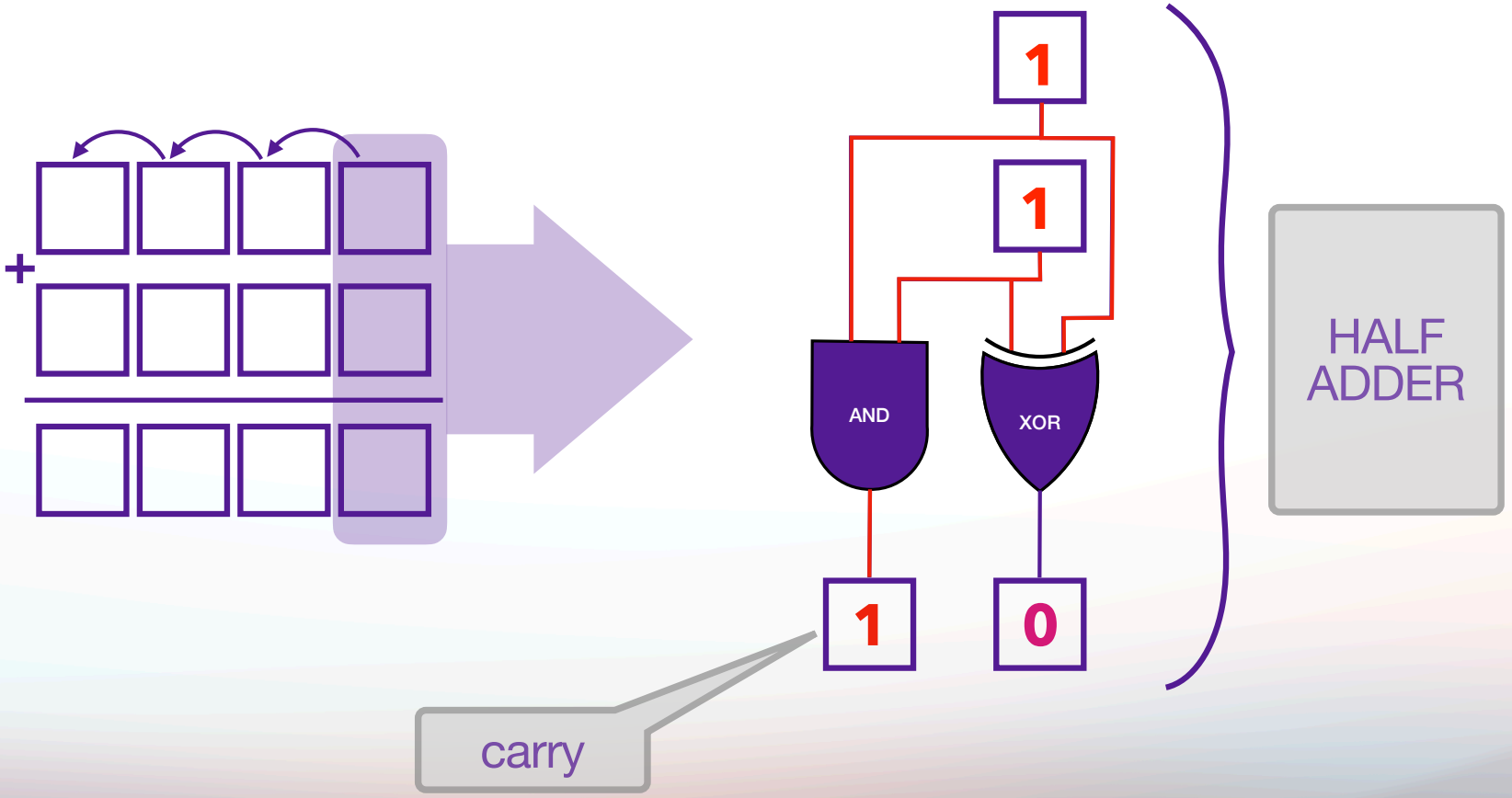
# Comencemos - Rehaciendo la PASCALINA con electricidad



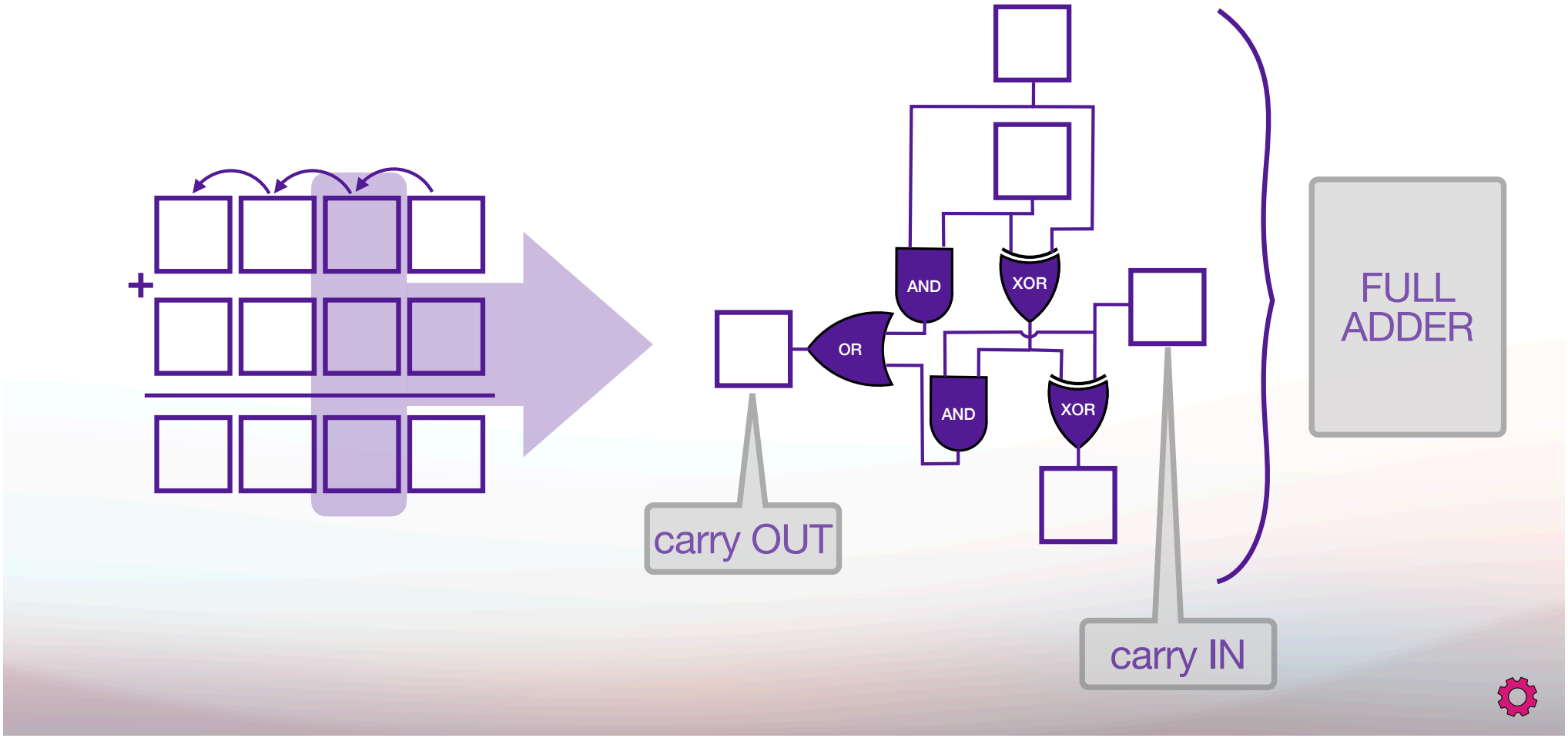
# Comencemos - Rehaciendo la PASCALINA con electricidad



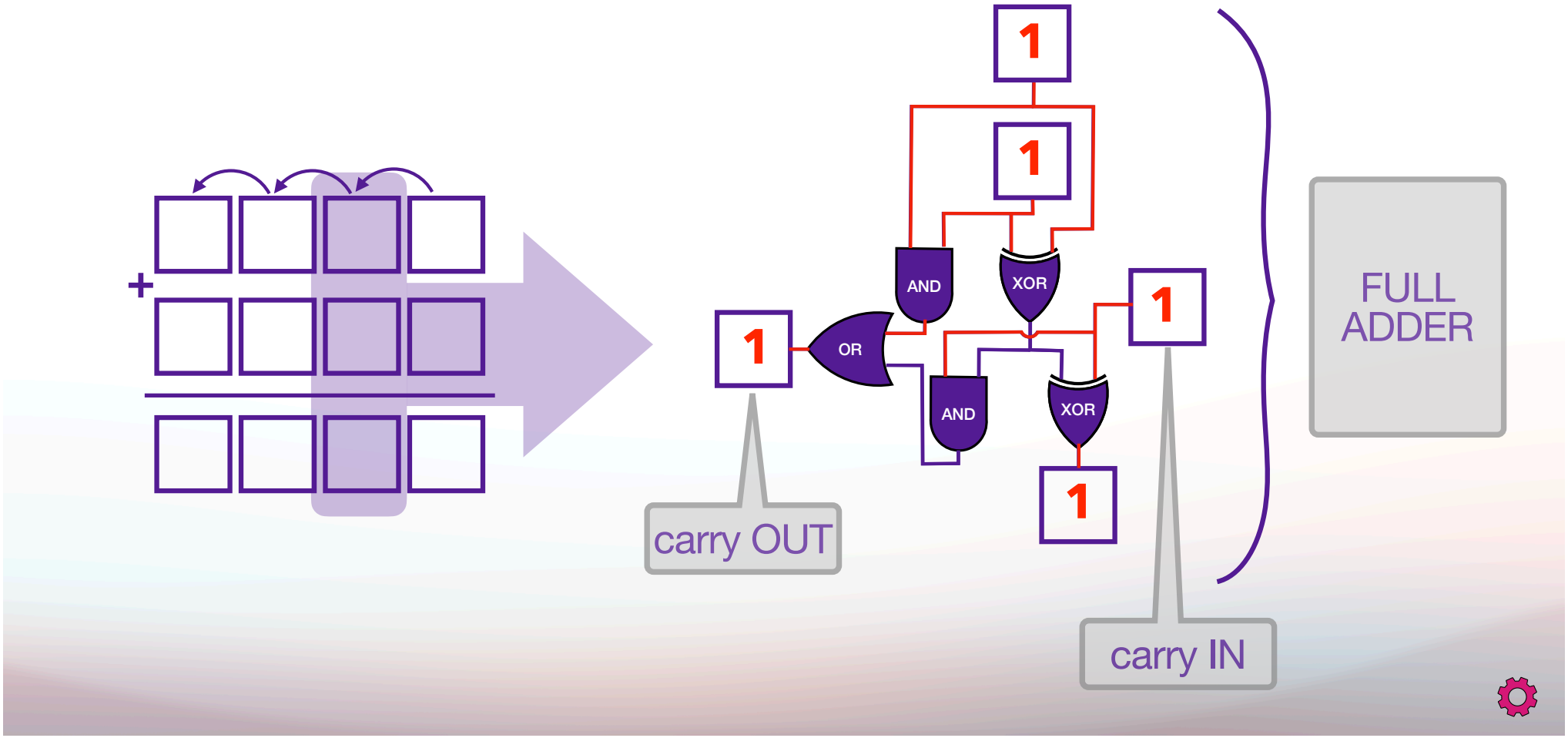
# Comencemos - Rehaciendo la PASCALINA con electricidad



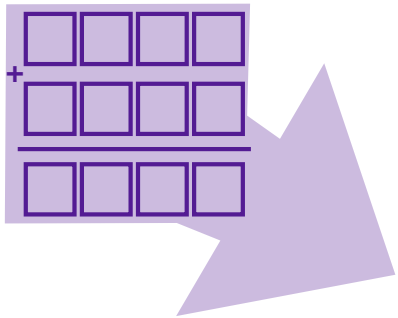
# Comencemos - Rehaciendo la PASCALINA con electricidad

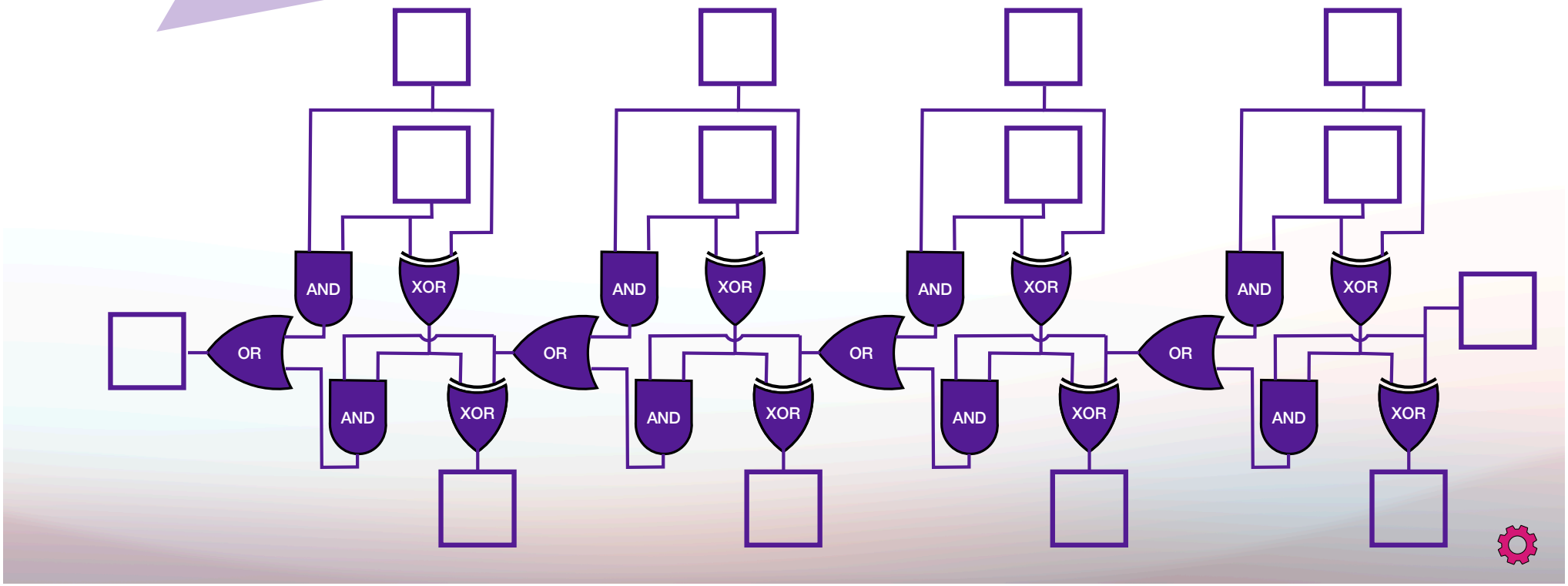


# Comencemos - Rehaciendo la PASCALINA con electricidad

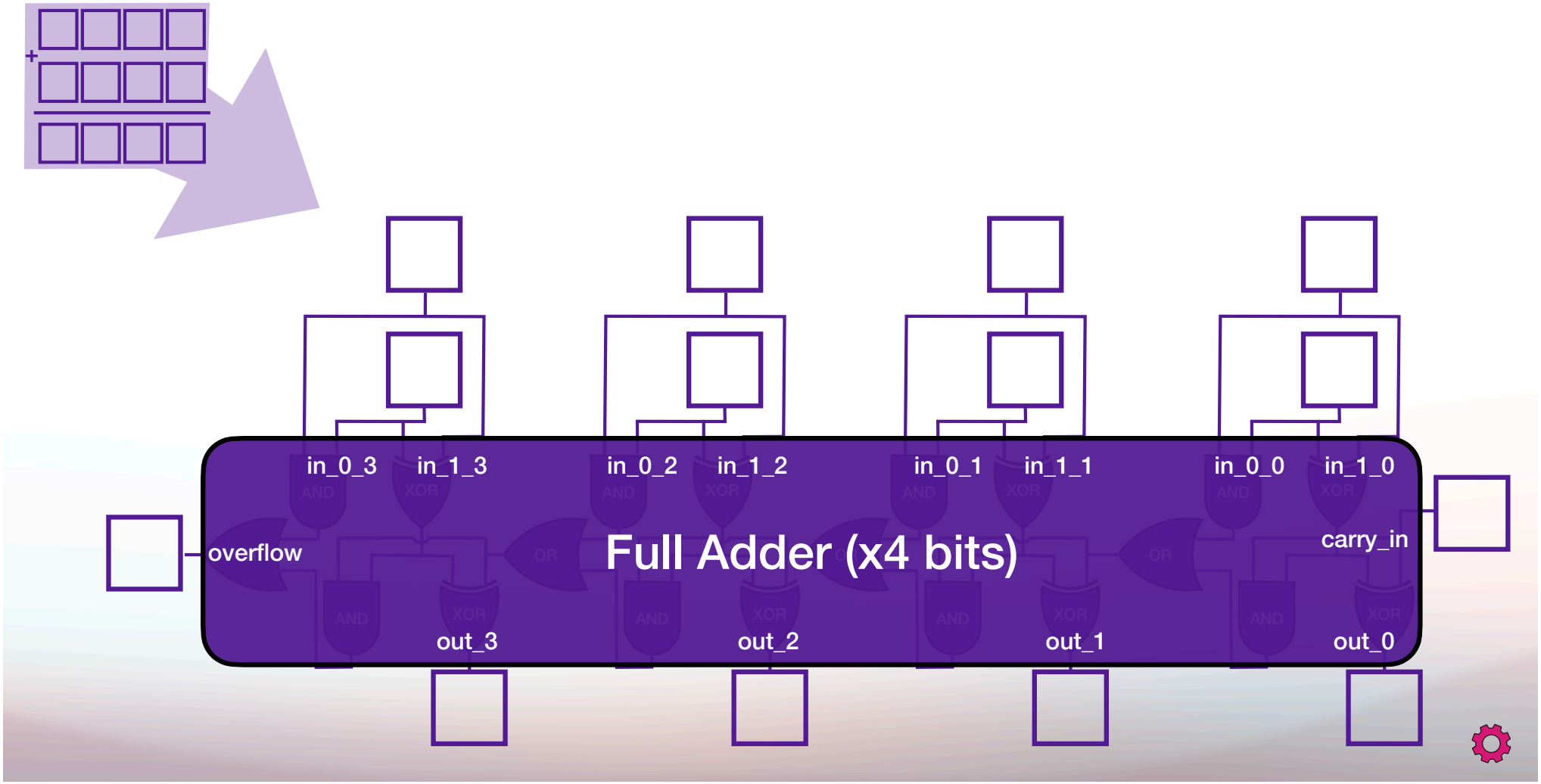


# Comencemos - Rehaciendo la PASCALINA con electricidad

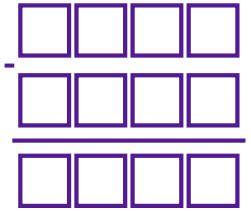


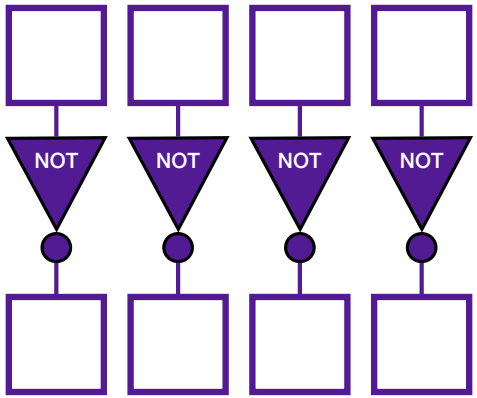


# Comencemos - Rehaciendo la PASCALINA con electricidad

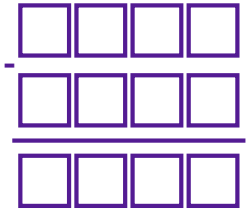


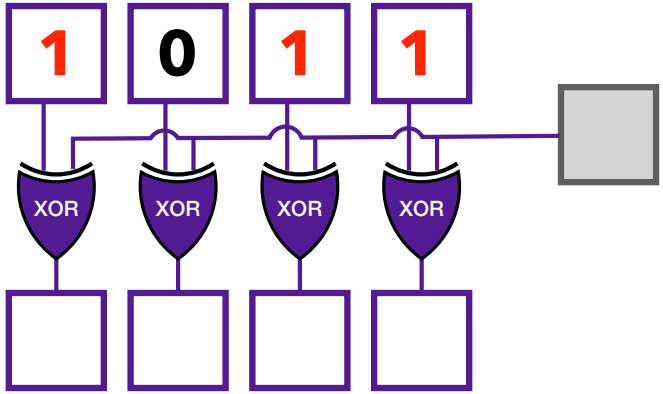
# Comencemos - ¿Cómo restamos? - Complemento



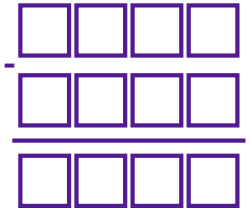


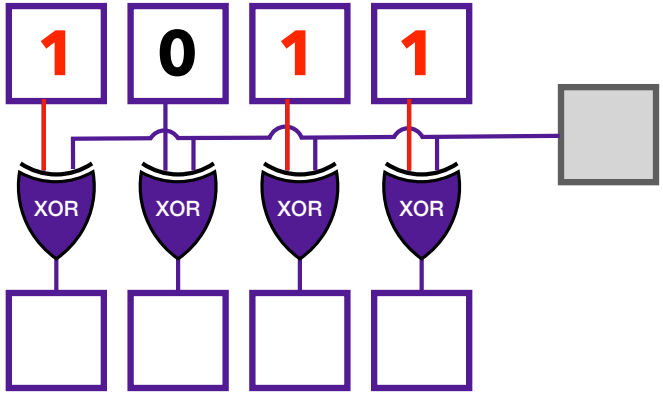
# Comencemos - ¿Cómo restamos? - Complemento



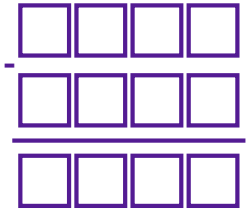


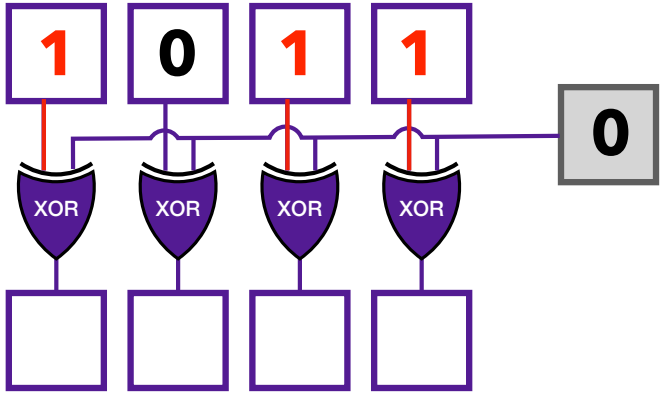
# Comencemos - ¿Cómo restamos? - Complemento



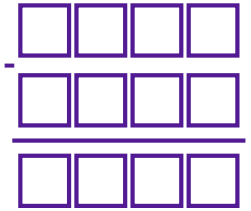


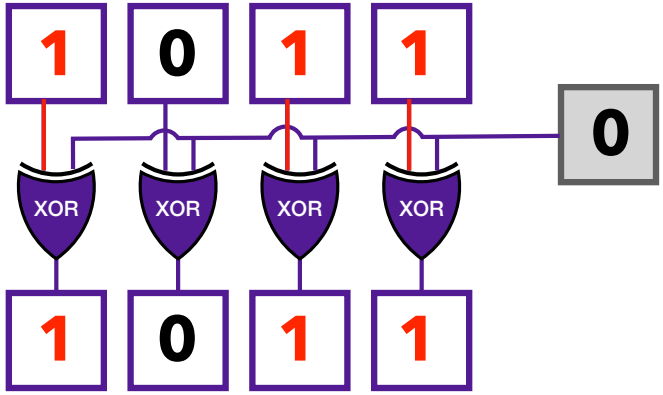
# Comencemos - ¿Cómo restamos? - Complemento



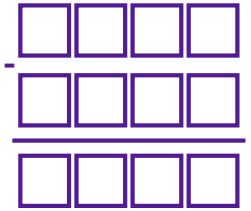


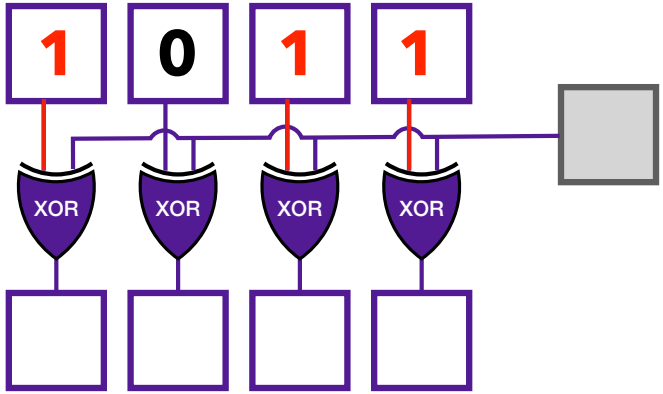
# Comencemos - ¿Cómo restamos? - Complemento



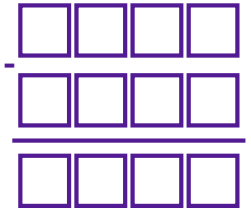


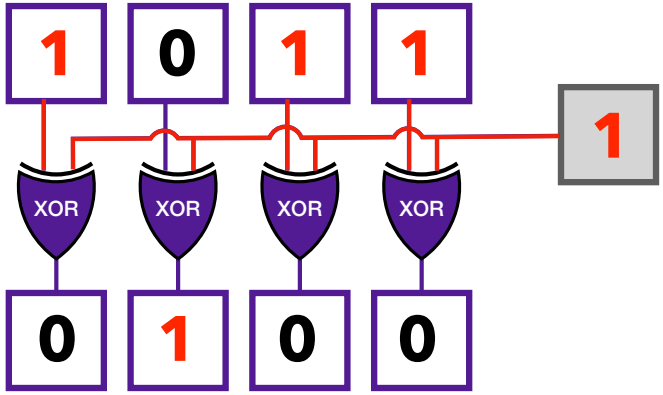
# Comencemos - ¿Cómo restamos? - Complemento



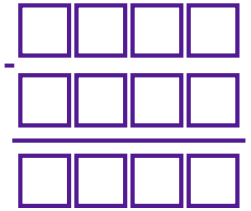


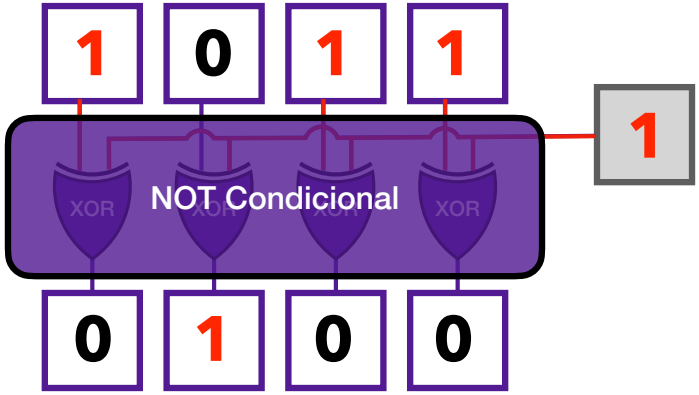
# Comencemos - ¿Cómo restamos? - Complemento



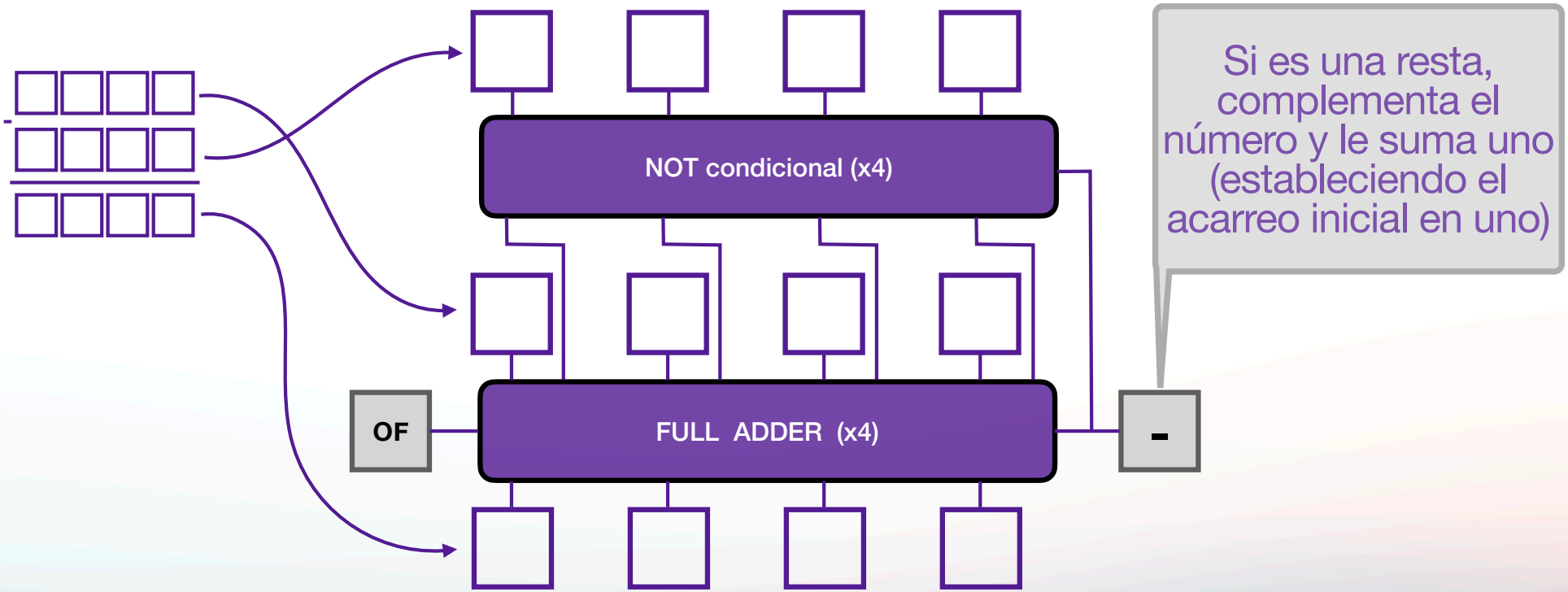


# Comencemos - ¿Cómo restamos? - Complemento

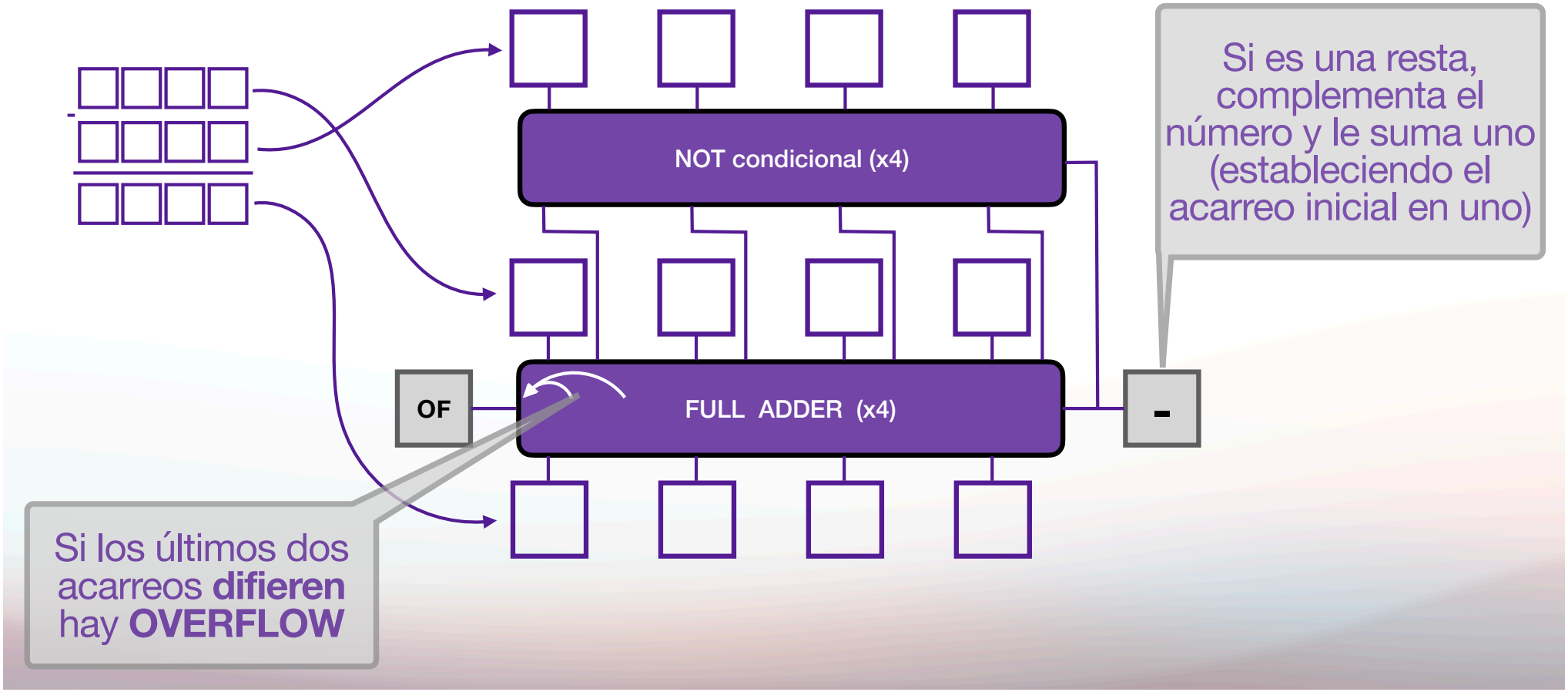




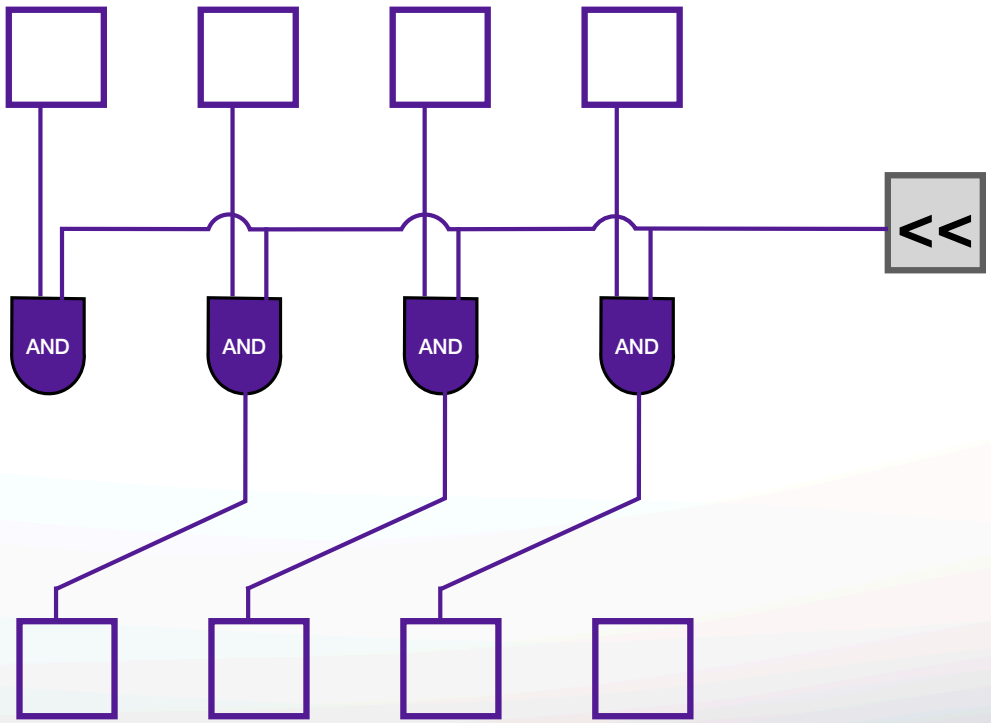
# Comencemos - ¿Cómo restamos? - Complemento



# Comencemos - ¿Cómo restamos? - Complemento



# Algunas operaciones necesarias - SHIFT (desplazar)



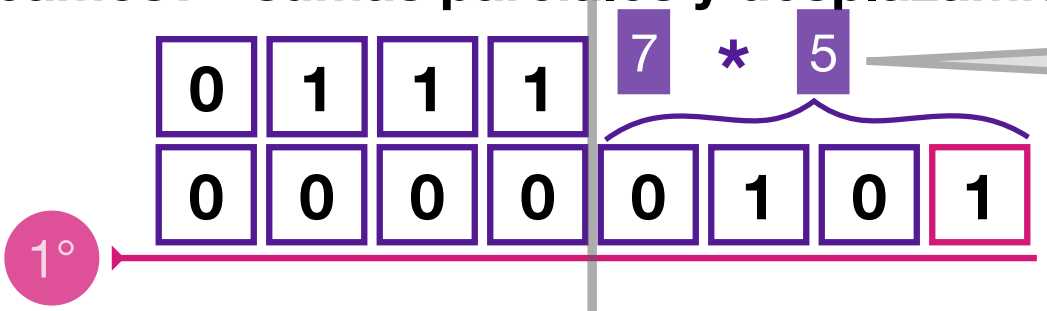
## ¿Cómo multiplicamos? - algunos tips

Podemos ir haciendo **sumas parciales del multiplicando**, cuando la posición del **multiplicador** que estamos tratando es 1. Luego en cualquier de los casos desplazamos.

El tamaño del resultado pude necesitar **hasta el doble de tamaño** de representación !

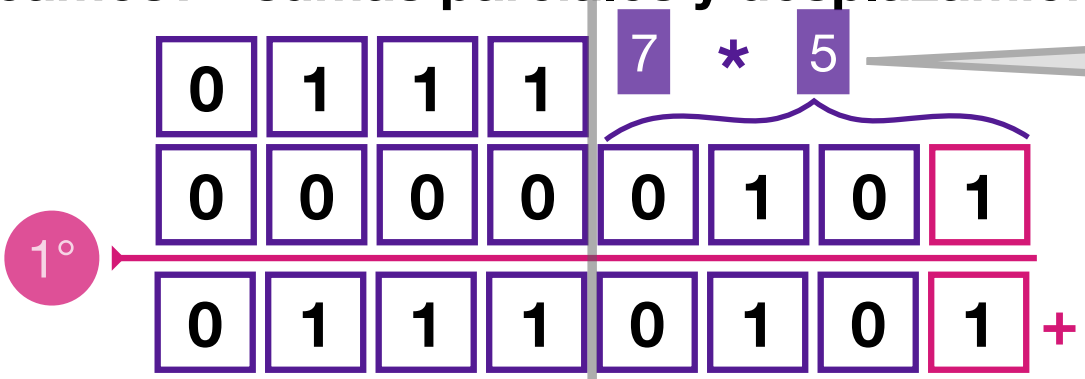
| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | 0 | 1 | 1 | 1 | 7 |
| | | | | * 0 | 1 | 0 | 1 | 5 |
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | | |
| 0 | 0 | 0 | 1 | 1 | 1 | | | |
| 0 | 0 | 0 | 0 | 0 | | | | |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 35 |

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento



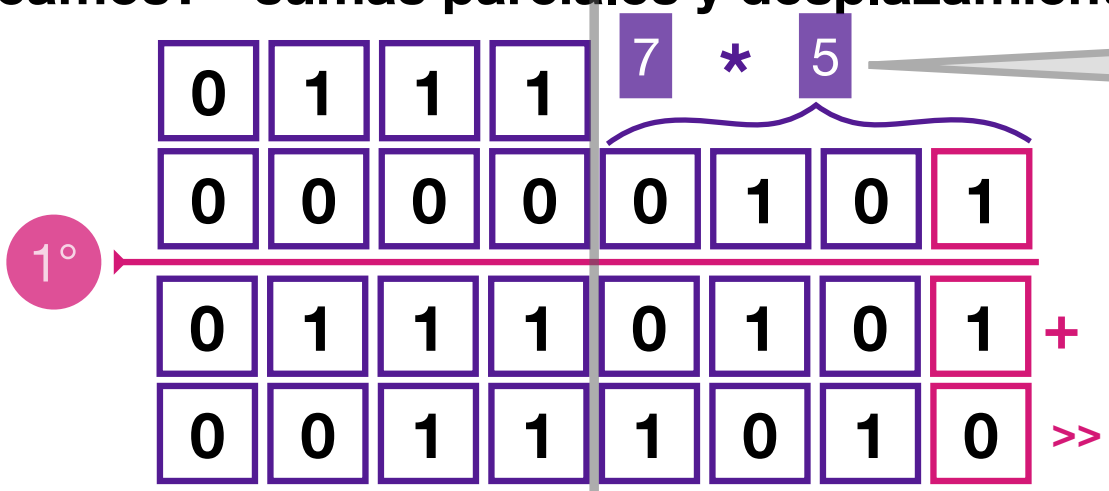

Como los bits del multiplicador, en la medida que los utilizo los puedo descartar, podemos utilizar ese **REGISTRO**, para almacenar el resultado de manera compuesta

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento



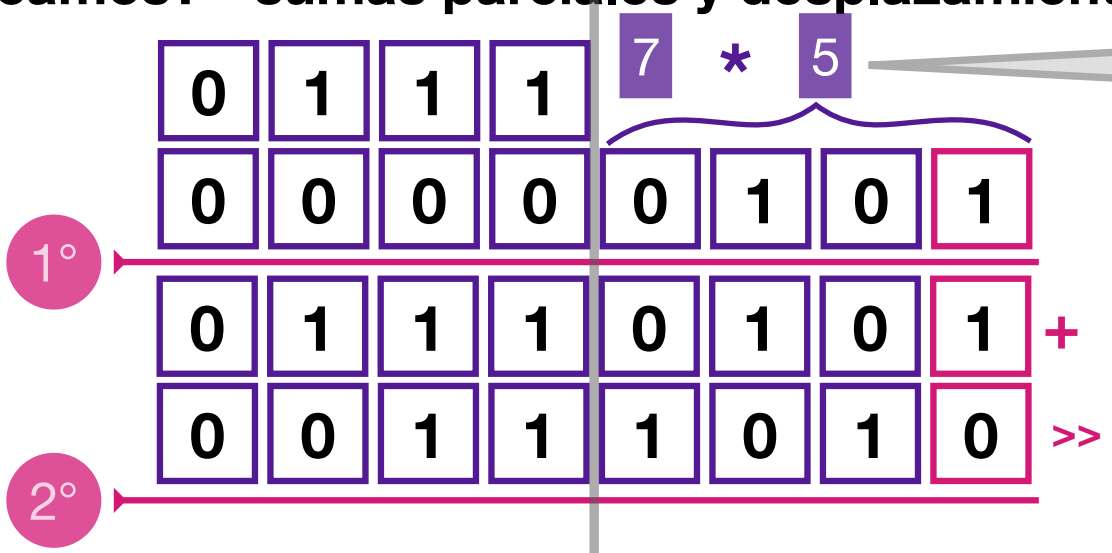

Como los bits del multiplicador, en la medida que los utilizo los puedo descartar, podemos utilizar ese **REGISTRO**, para almacenar el resultado de manera compuesta

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento



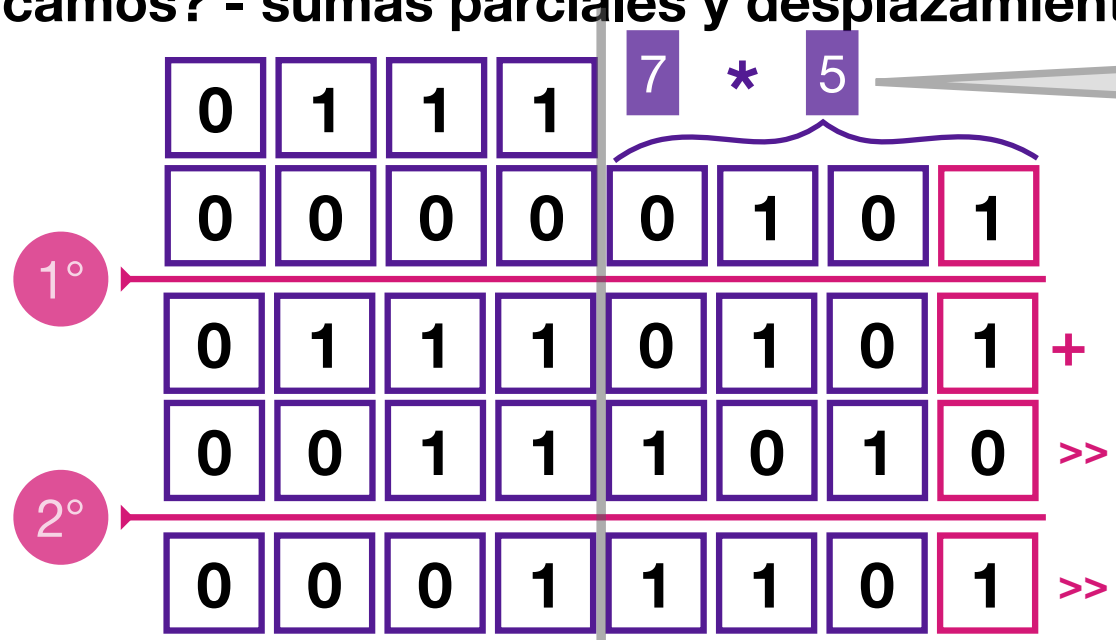

Como los bits del multiplicador, en la medida que los utilizo los puedo descartar, podemos utilizar ese **REGISTRO**, para almacenar el resultado de manera compuesta

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento



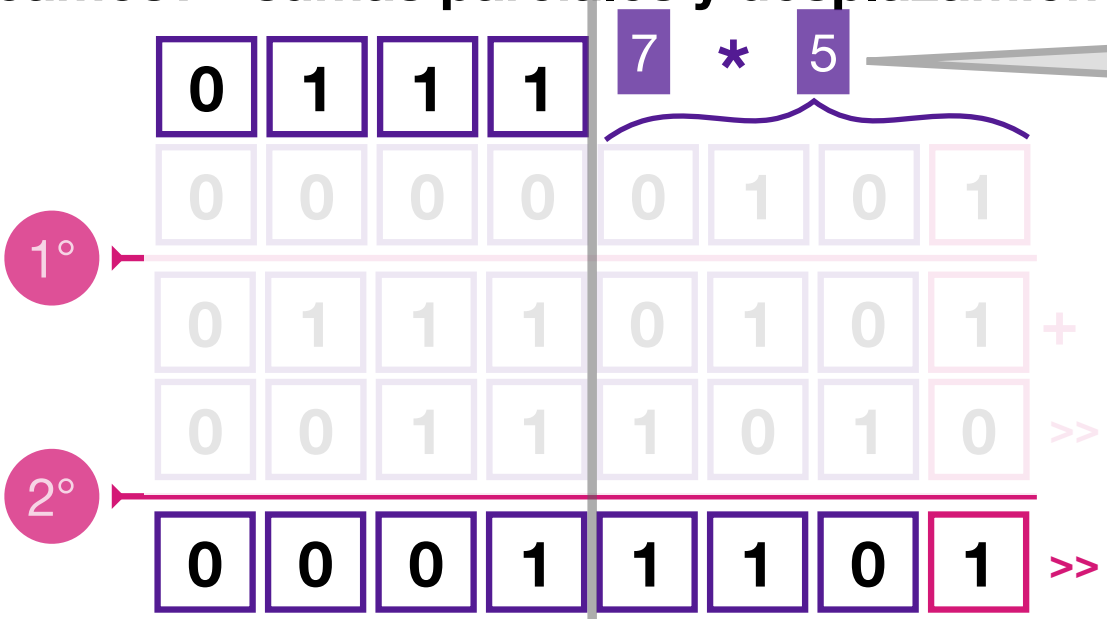

Como los bits del multiplicador, en la medida que los utilizo los puedo descartar, podemos utilizar ese **REGISTRO**, para almacenar el resultado de manera compuesta

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento



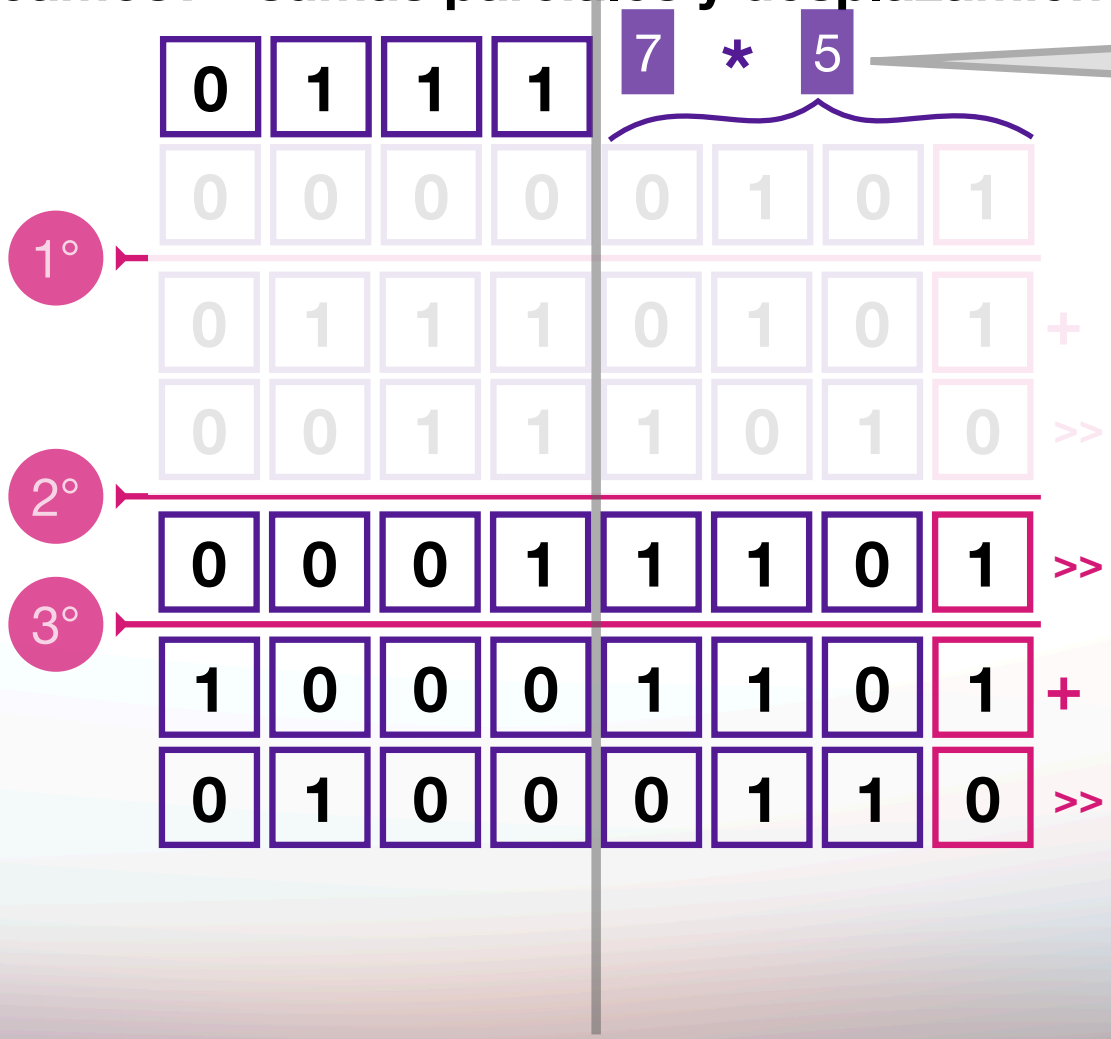

Como los bits del multiplicador, en la medida que los utilizo los puedo descartar, podemos utilizar ese **REGISTRO**, para almacenar el resultado de manera compuesta

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento



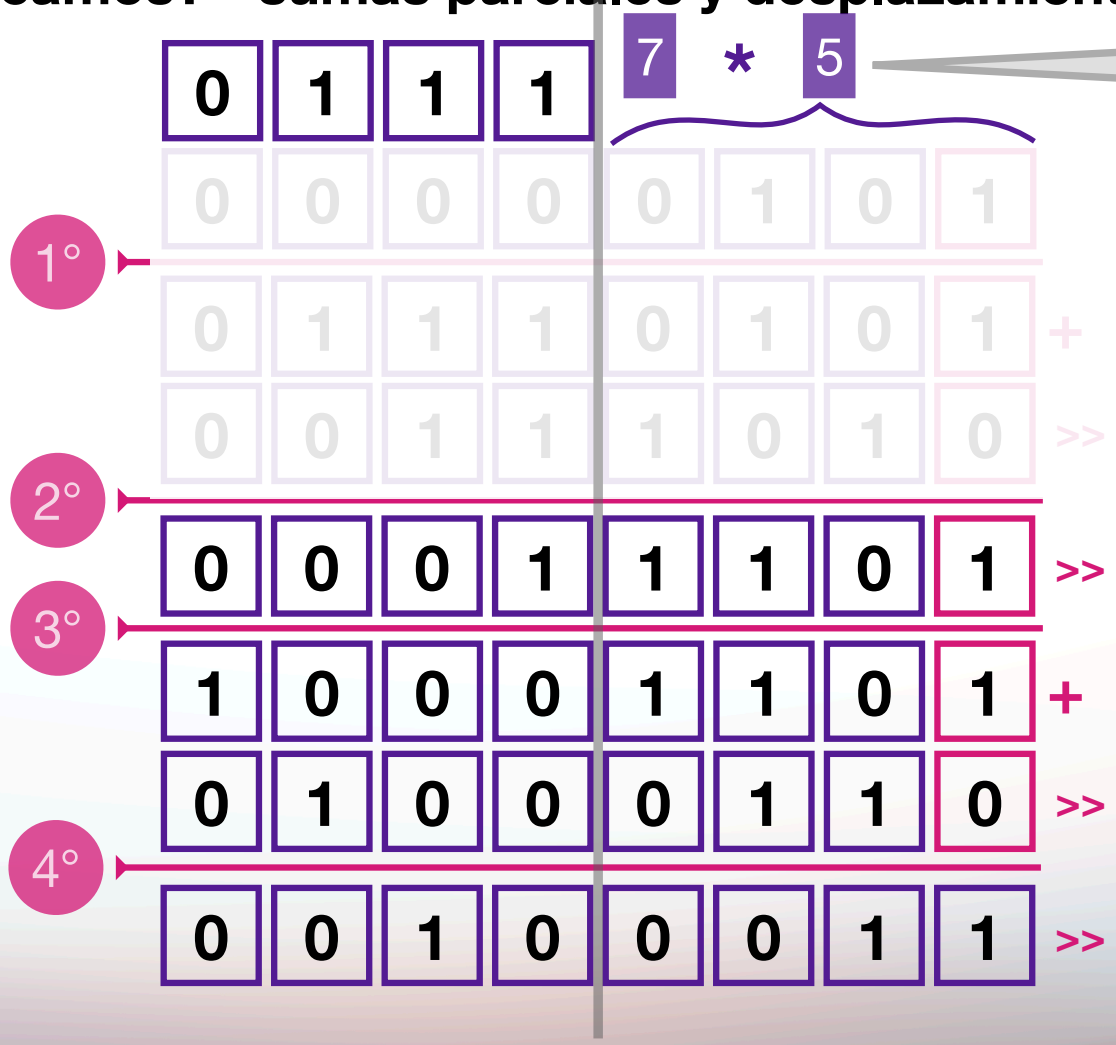

Como los bits del multiplicador, en la medida que los utilizo los puedo descartar, podemos utilizar ese **REGISTRO**, para almacenar el resultado de manera compuesta

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento



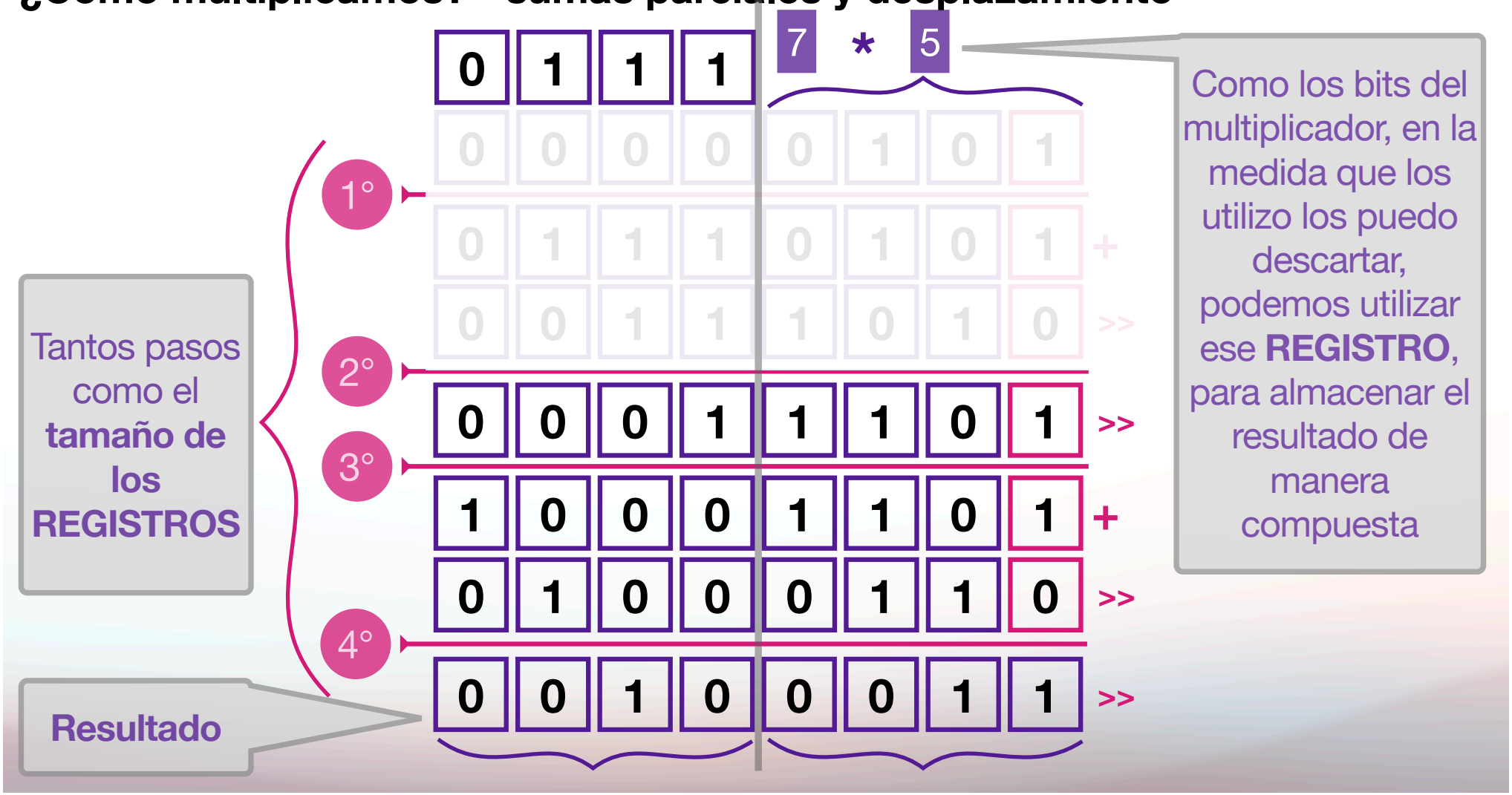

Como los bits del multiplicador, en la medida que los utilizo los puedo descartar, podemos utilizar ese **REGISTRO**, para almacenar el resultado de manera compuesta

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento




Como los bits del multiplicador, en la medida que los utilizo los puedo descartar, podemos utilizar ese **REGISTRO**, para almacenar el resultado de manera compuesta

## ¿Cómo multiplicamos? - sumas parciales y desplazamiento



## Algoritmo de la multiplicación

### Multiplicación de enteros general (positivos y negativos):

- El resultado necesita un tamaño de bits equivalente a la suma de los tamaños de los operandos.
- El proceso calcula productos parciales y depende de los siguientes casos:

(a)**+ Multiplicando * + Multiplicador** : Se comienza con el producto parcial igual a 0. Se recorre el multiplicador de derecha a izquierda. Si es un 0 se *shiftea* ***con el carry (igual al valor del primer bit del registro)*** *a derecha* el producto parcial. Si es un 1, al producto parcial se le suma el multiplicando y luego se *shiftea* ***con el carry out*** *a derecha* el producto parcial.

(b)**- Multiplicando * + Multiplicador** : Igual al proceso anterior.

(c)**+ Multiplicando * - Multiplicador** : se complementan ambos números y se procede como en (b)

(d)**- Multiplicando * - Multiplicador** : se complementan ambos números y se procede como en (a)

## Algoritmo de la multiplicación - Booth

La idea del algoritmo de Booth es la de no operar por casos y la de tratar de realizar la menor cantidad posible de sumas (es más costoso que *shiftear*).

Para ello el multiplicador que indica sumas (cuando es 1) o shirteos (cuando es 0) tiene un tratamiento especial siguiendo la siguiente fundamentación: cuando tenemos una secuencia de sumas (unos), su significado es equivalente a considerar la próxima posición a la más significativa menos la menos significativa.

**Ejemplo: 15**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | -1 |
| $2^9$ | $2^8$ | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |

**Ejemplo: 60**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | -1 | 0 | 0 |
| $2^9$ | $2^8$ | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |

**Ejemplo: 462**

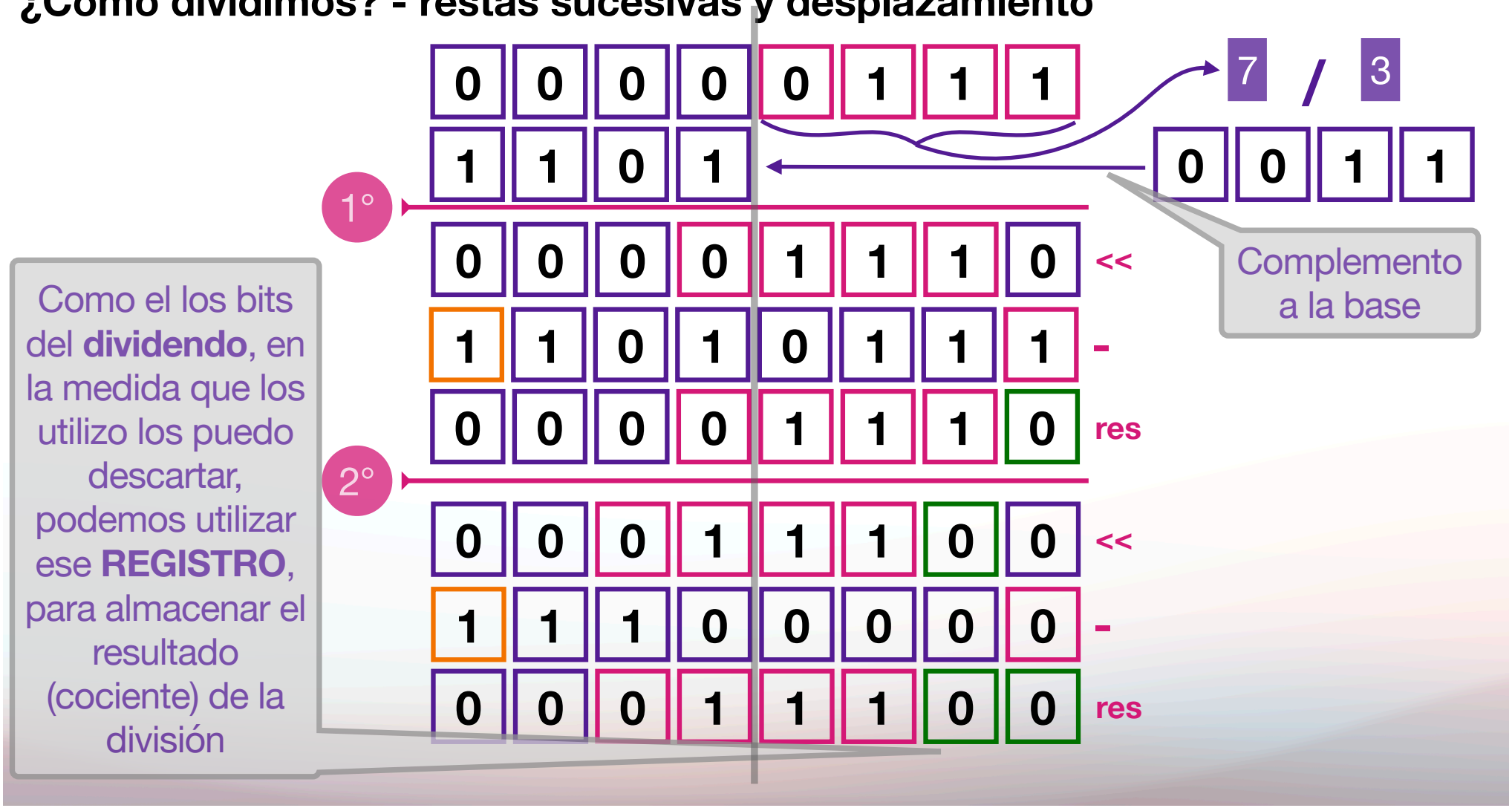
| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | -1 | 0 | 1 | 0 | 0 | -1 | 0 |
| $2^9$ | $2^8$ | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |

## Algoritmo de la multiplicación - Booth

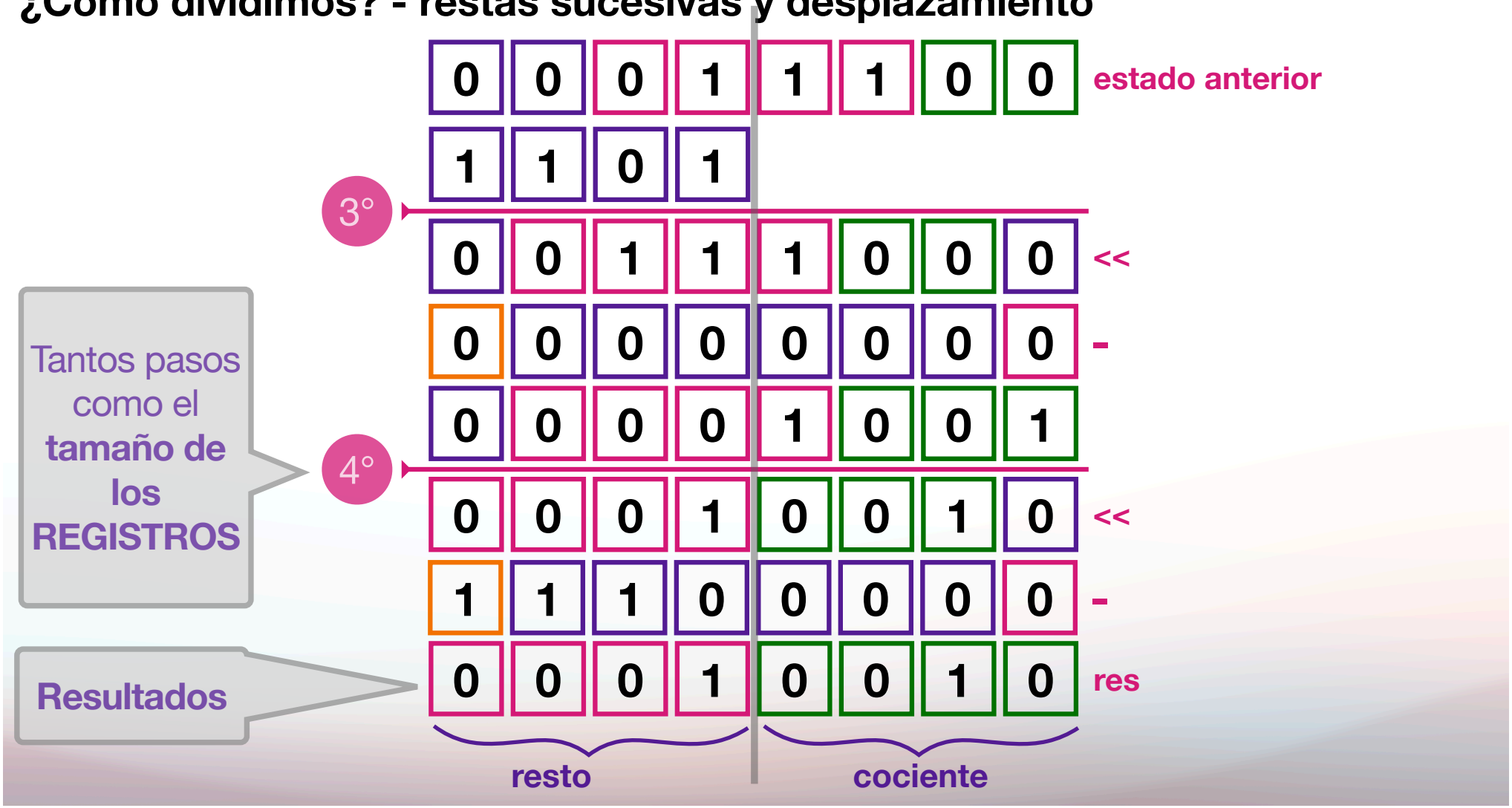
El resultado necesita un tamaño de bits equivalente a la suma del tamaño de los operandos.
El proceso calcula productos parciales de manera similar que el algoritmo general pero en vez de mirar el último bit del **multiplicador** tiene el cuenta **los últimos 2 bits** y de acuerdo a los siguientes casos opera sobre el producto parcial:

| | |
|---|---|
| 0 0 | → Se shiftea a derecha |
| 0 1 | → Se suma el multiplicando |
| 1 0 | → Se suma el complemento a la base (negativo) del multiplicando |
| 1 1 | → Se shiftea a derecha respetando la extensión de signo |

## ¿Cómo dividimos? - restas sucesivas y desplazamiento



## ¿Cómo dividimos? - restas sucesivas y desplazamiento



## Algoritmo de la división ( con restauración )

(1) *Shiftear a izquierda* una vez los registro correspondientes al **Dividendo** (Q) con **0** y el **registro auxiliar** (R) con el bit más significativo de Q.

(2) Dejar en el registro auxiliar el resultado de restarle al mismo el **Divisor** (S). Notar que se suma el Comp.Base de (S)

(3) Si el signo (***bit más*** significativo del **registo auxiliar** R) es 1 (negativo), cambiar el ***bit menos*** significativo de **Dividendo** ($Q_0$) a 0 y restaurar el **registo auxiliar** sumando el **Divisor** (S), sino cambiar el ***bit menos*** significativo de **Dividendo** ($Q_0$) a 1.